```
// FILE: src/main/java/com/sqlteacher/infrastructure/database/DefaultSqlRiskAnalysisService.java
package com.sqlteacher.infrastructure.database;

import com.sqlteacher.application.connection.DatabaseDialect;
import com.sqlteacher.application.risk.SqlRiskAnalysis;
import com.sqlteacher.application.risk.SqlRiskAnalysisService;
import com.sqlteacher.application.risk.SqlRiskLevel;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.Objects;
import java.util.regex.Pattern;

public final class DefaultSqlRiskAnalysisService implements SqlRiskAnalysisService {
    @Override
    public SqlRiskAnalysis analyze(String sql) {
        return analyze(sql, DatabaseDialect.SQLITE);
    }

    @Override
    public SqlRiskAnalysis analyze(String sql, DatabaseDialect dialect) {
        Objects.requireNonNull(dialect, "dialect must not be null");

        if (sql == null || sql.isBlank()) {
            return forbidden("UNKNOWN", false, "SQL must not be blank");
        }

        String normalized = removeComments(sql).strip();

        if (normalized.isBlank()) {
            return forbidden("UNKNOWN", false, "SQL must contain a statement");
        }

        boolean multiStatement = hasMultipleStatements(normalized);
        String statementType = firstKeyword(normalized);

        if (multiStatement) {
            return new SqlRiskAnalysis(
                    SqlRiskLevel.HIGH,
                    false,
                    true,
                    true,
                    statementType,
                    List.of("Multiple SQL statements are not allowed.")
            );
        }

        if (containsAiSeparatedStatement(normalized)) {
            return new SqlRiskAnalysis(
```

```
            SqlRiskLevel.HIGH,
            false,
            true,
            true,
            "MULTI_STATEMENT",
            List.of("Multiple SQL statements detected. Execute only one statement at a time.")
    );
}

SqlRiskAnalysis dialectRisk = analyzeDialectSpecificRisk(normalized, statementType, dialect);
if (dialectRisk != null) {
    return dialectRisk;
}

if (statementType.equals("DROP")
        && normalized.toUpperCase(Locale.ROOT).matches("DROP\\s+DATABASE\\b.*")) {
    return forbidden(
            "DROP",
            false,
            "DROP DATABASE is not allowed."
    );
}

return switch (statementType) {

    case "SELECT" -> new SqlRiskAnalysis(
            SqlRiskLevel.LOW,
            true,
            false,
            false,
            statementType,
            List.of("Read-only query.")
    );

    case "INSERT", "UPDATE", "DELETE" -> new SqlRiskAnalysis(
            SqlRiskLevel.MEDIUM,
            true,
            true,
            false,
            statementType,
            List.of("This statement modifies data.")
    );

    case "CREATE", "ALTER", "DROP", "TRUNCATE" -> new SqlRiskAnalysis(
            SqlRiskLevel.HIGH,
            true,
            true,
            false,
            statementType,
            List.of("This statement modifies database schema.")
```

```
        );

        default -> forbidden(
                statementType,
                false,
                "Unsupported SQL statement."
        );
    };
}

private SqlRiskAnalysis analyzeDialectSpecificRisk(
        String sql,
        String statementType,
        DatabaseDialect dialect
) {
    if ((dialect != DatabaseDialect.MYSQL && dialect != DatabaseDialect.MARIADB)
            || !"SELECT".equals(statementType)) {
        return null;
    }
    String tokens = maskQuotedText(sql).toUpperCase(Locale.ROOT);
    if (MYSQL_FILE_OUTPUT.matcher(tokens).find()) {
        return forbidden("SELECT", false, "MySQL file output is not allowed.");
    }
    if (MYSQL_LOCKING_SELECT.matcher(tokens).find()) {
        return forbidden("SELECT", false, "MySQL locking queries are not allowed.");
    }
    if (MYSQL_DANGEROUS_FUNCTION.matcher(tokens).find()) {
        return forbidden("SELECT", false, "MySQL file, lock, or delay functions are not allowed.");
    }
    return null;
}

private SqlRiskAnalysis forbidden(
        String statementType,
        boolean multiStatement,
        String reason
) {

    List<String> reasons = new ArrayList<>();
    reasons.add(reason);

    return new SqlRiskAnalysis(
            SqlRiskLevel.FORBIDDEN,
            false,
            false,
            multiStatement,
            statementType,
            reasons
    );
}
```

```
private static final Pattern AI_MULTI_STATEMENT = Pattern.compile(
        "(?is)\\R\\s*\\R\\s*(SELECT|INSERT|UPDATE|DELETE|DROP|ALTER|CREATE|TRUNCATE)\\b"
);
private static final Pattern MYSQL_FILE_OUTPUT = Pattern.compile("\\bINTO\\s+(OUTFILE|DUMPFILE)\\b");
private static final Pattern MYSQL_LOCKING_SELECT = Pattern.compile(
        "\\bFOR\\s+UPDATE\\b|\\bLOCK\\s+IN\\s+SHARE\\s+MODE\\b"
);
private static final Pattern MYSQL_DANGEROUS_FUNCTION = Pattern.compile(
        "\\b(SLEEP|BENCHMARK|GET_LOCK|RELEASE_LOCK|LOAD_FILE)\\s*\\("
);

private boolean containsAiSeparatedStatement(String sql) {
    return AI_MULTI_STATEMENT.matcher(sql).find();
}

private boolean hasMultipleStatements(String sql) {
    boolean singleQuoted = false;
    boolean doubleQuoted = false;
    boolean lineComment = false;
    boolean blockComment = false;

    for (int index = 0; index < sql.length(); index++) {
        char current = sql.charAt(index);
        char next = index + 1 < sql.length() ? sql.charAt(index + 1) : '\0';

        if (lineComment) {
            lineComment = current != '\n' && current != '\r';
            continue;
        }
        if (blockComment) {
            if (current == '*' && next == '/') {
                blockComment = false;
                index++;
            }
            continue;
        }
        if (!singleQuoted && !doubleQuoted && current == '-' && next == '-') {
            lineComment = true;
            index++;
            continue;
        }
        if (!singleQuoted && !doubleQuoted && current == '/' && next == '*') {
            blockComment = true;
            index++;
            continue;
        }
        if (!doubleQuoted && current == '\'') {
            if (singleQuoted && next == '\'') {
                index++;
```

```
            } else {
                singleQuoted = !singleQuoted;
            }
            continue;
        }
        if (!singleQuoted && current == '"') {
            if (doubleQuoted && next == '"') {
                index++;
            } else {
                doubleQuoted = !doubleQuoted;
            }
            continue;
        }
        if (!singleQuoted && !doubleQuoted && current == ';'
                && hasStatementContent(sql, index + 1)) {
            return true;
        }
    }
    return false;
}

private boolean hasStatementContent(String sql, int start) {
    String remainder = sql.substring(start)
            .replaceAll("(?s)/\\*.*?\\*/", "")
            .replaceAll("--[^\\r\\n]*", "")
            .replace(";", "")
            .strip();
    return !remainder.isEmpty();
}

private String firstKeyword(String sql) {

    int index = 0;

    while (index < sql.length() && Character.isLetter(sql.charAt(index))) {
        index++;
    }

    if (index == 0) {
        return "UNKNOWN";
    }

    return sql.substring(0, index).toUpperCase(Locale.ROOT);
}

private String removeComments(String sql) {
    StringBuilder normalized = new StringBuilder(sql.length());
    boolean singleQuoted = false;
    boolean doubleQuoted = false;
    boolean backtickQuoted = false;
```

```
boolean bracketQuoted = false;

for (int index = 0; index < sql.length(); index++) {
    char current = sql.charAt(index);
    char next = index + 1 < sql.length() ? sql.charAt(index + 1) : '\0';

    if (!singleQuoted && !doubleQuoted && !backtickQuoted && !bracketQuoted
            && current == '-' && next == '-') {
        normalized.append(' ');
        index += 2;
        while (index < sql.length() && sql.charAt(index) != '\n' && sql.charAt(index) != '\r') {
            index++;
        }
        if (index < sql.length()) {
            normalized.append(sql.charAt(index));
        }
        continue;
    }
    if (!singleQuoted && !doubleQuoted && !backtickQuoted && !bracketQuoted
            && current == '/' && next == '*') {
        normalized.append(' ');
        index += 2;
        while (index < sql.length()) {
            char commentCurrent = sql.charAt(index);
            char commentNext = index + 1 < sql.length() ? sql.charAt(index + 1) : '\0';
            if (commentCurrent == '*' && commentNext == '/') {
                index++;
                break;
            }
            if (commentCurrent == '\n' || commentCurrent == '\r') {
                normalized.append(commentCurrent);
            }
            index++;
        }
        normalized.append(' ');
        continue;
    }

    normalized.append(current);

    if (singleQuoted && current == '\'' && next == '\'') {
        normalized.append(next);
        index++;
    } else if (doubleQuoted && current == '"' && next == '"') {
        normalized.append(next);
        index++;
    } else if (backtickQuoted && current == '`' && next == '`') {
        normalized.append(next);
        index++;
    } else if (!doubleQuoted && !backtickQuoted && !bracketQuoted && current == '\'') {
```

```
                singleQuoted = !singleQuoted;
            } else if (!singleQuoted && !backtickQuoted && !bracketQuoted && current == '"') {
                doubleQuoted = !doubleQuoted;
            } else if (!singleQuoted && !doubleQuoted && !bracketQuoted && current == '`') {
                backtickQuoted = !backtickQuoted;
            } else if (!singleQuoted && !doubleQuoted && !backtickQuoted && current == '[') {
                bracketQuoted = true;
            } else if (bracketQuoted && current == ']') {
                bracketQuoted = false;
            }
        }

        return normalized.toString();
    }

    private static String maskQuotedText(String sql) {
        StringBuilder masked = new StringBuilder(sql.length());
        char quote = '\0';
        for (int index = 0; index < sql.length(); index++) {
            char current = sql.charAt(index);
            char next = index + 1 < sql.length() ? sql.charAt(index + 1) : '\0';
            if (quote == '\0' && (current == '\'' || current == '"' || current == '`')) {
                quote = current;
                masked.append(' ');
            } else if (quote != '\0') {
                masked.append(' ');
                if (current == quote && next == quote) {
                    masked.append(' ');
                    index++;
                } else if (current == quote && (index == 0 || sql.charAt(index - 1) != '\\')) {
                    quote = '\0';
                }
            } else {
                masked.append(current);
            }
        }
        return masked.toString();
    }
}
// FILE: src/main/java/com/sqlteacher/infrastructure/database/SqlScriptSplitter.java
package com.sqlteacher.infrastructure.database;

import java.util.ArrayList;
import java.util.List;

final class SqlScriptSplitter {
    private SqlScriptSplitter() {
    }

    static List<String> split(String script) {
```

```
List<String> statements = new ArrayList<>();
StringBuilder current = new StringBuilder();
char quote = '\0';
boolean lineComment = false;
boolean blockComment = false;

for (int index = 0; index < script.length(); index++) {
    char character = script.charAt(index);
    char next = index + 1 < script.length() ? script.charAt(index + 1) : '\0';
    if (lineComment) {
        current.append(character);
        if (character == '\n' || character == '\r') {
            lineComment = false;
        }
        continue;
    }
    if (blockComment) {
        current.append(character);
        if (character == '*' && next == '/') {
            current.append(next);
            index++;
            blockComment = false;
        }
        continue;
    }
    if (quote == '\0' && character == '-' && next == '-') {
        current.append(character).append(next);
        index++;
        lineComment = true;
        continue;
    }
    if (quote == '\0' && character == '/' && next == '*') {
        current.append(character).append(next);
        index++;
        blockComment = true;
        continue;
    }
    if (quote == '\0' && (character == '\'' || character == '"' || character == '`')) {
        quote = character;
        current.append(character);
        continue;
    }
    if (quote != '\0' && character == quote) {
        current.append(character);
        if (next == quote) {
            current.append(next);
            index++;
        } else {
            quote = '\0';
        }
```

```
                continue;
            }
            if (quote == '\0' && character == ';') {
                addStatement(statements, current);
            } else {
                current.append(character);
            }
        }
        addStatement(statements, current);
        return List.copyOf(statements);
    }

    private static void addStatement(List<String> statements, StringBuilder current) {
        String statement = current.toString().trim();
        if (!statement.isEmpty()) {
            statements.add(statement);
        }
        current.setLength(0);
    }
}
// FILE: src/main/java/com/sqlteacher/infrastructure/database/JdbcSqlExecutionService.java
package com.sqlteacher.infrastructure.database;

import com.sqlteacher.application.execution.SqlExecutionRequest;
import com.sqlteacher.application.execution.SqlExecutionResult;
import com.sqlteacher.application.execution.SqlExecutionService;
import com.sqlteacher.application.event.LearningEventService;
import com.sqlteacher.application.risk.SqlRiskAnalysis;
import com.sqlteacher.application.risk.SqlRiskAnalysisService;
import com.sqlteacher.domain.SqlTeacherException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.time.Duration;
import java.time.Instant;
import java.util.Objects;

public final class JdbcSqlExecutionService implements SqlExecutionService {
    private final JdbcConnectionProvider connectionProvider;
    private final SqlResultMapper resultMapper;
    private final SqlRiskAnalysisService riskAnalysisService;
    private final LearningEventService eventService;
    private static final Logger log = LoggerFactory.getLogger(JdbcSqlExecutionService.class);

    public JdbcSqlExecutionService(
            JdbcConnectionProvider connectionProvider,
```

```
        SqlResultMapper resultMapper,
        SqlRiskAnalysisService riskAnalysisService,
        LearningEventService eventService
) {
    this.connectionProvider = Objects.requireNonNull(connectionProvider);
    this.resultMapper = Objects.requireNonNull(resultMapper);
    this.riskAnalysisService = Objects.requireNonNull(riskAnalysisService);
    this.eventService = Objects.requireNonNull(eventService);
}

@Override
public SqlExecutionResult execute(SqlExecutionRequest request) {

    validate(request);

     log.debug("Executing SQL on connection '{}', sqlLength={}",
            request.connectionId(),
            request.sql().length()
    );

    SqlRiskAnalysis risk = riskAnalysisService.analyze(
        request.sql(),
        connectionProvider.dialect(request.connectionId())
    );

    log.debug("Risk analysis result: level={}, executable={}, confirmationRequired={}, type={}",
            risk.level(),
            risk.executable(),
            risk.confirmationRequired(),
            risk.statementType()
    );

    if (!risk.executable()) {
        // Record risk blocked event
        eventService.recordSqlRiskBlocked(
                request.connectionId(),
                risk.statementType(),
                risk.level(),
                risk.multiStatement()
        );

        throw new SqlTeacherException(
                "SQL_BLOCKED",
                risk.reasons().isEmpty()
                        ? "SQL execution blocked."
                        : risk.reasons().getFirst()
        );
    }

    if (connectionProvider.isReadOnly(request.connectionId()) && !"SELECT".equals(risk.statementType())) {
```

```
    eventService.recordSqlRiskBlocked(
        request.connectionId(),
        risk.statementType(),
        risk.level(),
        risk.multiStatement()
    );
    throw new SqlTeacherException(
        "SQL_READ_ONLY_CONNECTION",
        "当前数据库连接为只读模式，只允许执行 SELECT 查询。"
    );
}

if (risk.confirmationRequired() && !request.riskConfirmed()) {
    eventService.recordSqlRiskBlocked(
            request.connectionId(),
            risk.statementType(),
            risk.level(),
            risk.multiStatement()
    );

    throw new SqlTeacherException(
            "SQL_CONFIRMATION_REQUIRED",
            "This SQL requires user confirmation before execution."
    );
}

Instant start = Instant.now();

log.debug("Starting SQL execution");

try (
        Connection connection =
                connectionProvider.open(request.connectionId(), request.timeout());

        Statement statement =
                connection.createStatement()
) {

    statement.setQueryTimeout(
            (int) request.timeout().toSeconds()
    );

    statement.setMaxRows(queryProbeLimit(request.maxRows()));

    boolean hasResult =
            statement.execute(request.sql());

    Duration duration =
            Duration.between(start, Instant.now());
```

```
    if (hasResult) {

        try (ResultSet resultSet = statement.getResultSet()) {
            log.debug("Query execution completed in {} ms", duration.toMillis());
            SqlExecutionResult result = resultMapper.mapQueryResult(
                    resultSet,
                    duration,
                    request.maxRows()
            );

            // Record successful SQL execution event
            eventService.recordSqlExecution(
                    request.connectionId(),
                    true,
                    risk.statementType(),
                    duration,
                    result.rows().size(),
                    null
            );

            return result;

        }

    }

    log.debug("Update execution completed in {} ms, affectedRows={}", duration.toMillis(), statement.getUpdateCount());
    SqlExecutionResult result = resultMapper.mapUpdateResult(
            statement.getUpdateCount(),
            duration
    );

    // Record successful SQL execution event for updates
    eventService.recordSqlExecution(
            request.connectionId(),
            true,
            risk.statementType(),
            duration,
            result.affectedRows(),
            null
    );

    return result;

} catch (SQLException exception) {
    JdbcFailureClassifier.JdbcFailure failure = JdbcFailureClassifier.classify(exception);
    log.warn("SQL execution failed, connectionId={}, failureType={}, sqlState={}, vendorCode={}",
            request.connectionId(),
            failure,
            JdbcFailureClassifier.sqlState(exception),
```

```
                JdbcFailureClassifier.vendorCode(exception)
        );

        // Record failed SQL execution event
        Duration duration = Duration.between(start, Instant.now());
        eventService.recordSqlExecution(
                request.connectionId(),
                false,
                risk.statementType(),
                duration,
                0,
                failure.errorCode()
        );

        throw new SqlTeacherException(
                failure.errorCode(),
                failure.userMessage()
        );

    }

}

private static int queryProbeLimit(int maxRows) {
    return maxRows == Integer.MAX_VALUE ? Integer.MAX_VALUE : maxRows + 1;
}

private static void validate(SqlExecutionRequest request) {

    Objects.requireNonNull(request, "request must not be null");

    if (request.connectionId() == null ||
            request.connectionId().isBlank()) {

        throw new IllegalArgumentException(
                "connectionId must not be blank"
        );
    }

    if (request.sql() == null ||
            request.sql().isBlank()) {

        throw new IllegalArgumentException(
                "sql must not be blank"
        );
    }

    if (request.maxRows() <= 0) {

        throw new IllegalArgumentException(
```

```
                    "maxRows must be positive"
            );
        }

        if (request.timeout() == null ||
                request.timeout().isNegative() ||
                request.timeout().isZero()) {

            throw new IllegalArgumentException(
                    "timeout must be positive"
            );
        }

    }

}
// FILE: src/main/java/com/sqlteacher/infrastructure/database/DeterministicSqlExerciseEvaluationService.java
package com.sqlteacher.infrastructure.database;

import com.sqlteacher.application.config.SqlTeacherConfiguration;
import com.sqlteacher.application.connection.DatabaseDialect;
import com.sqlteacher.application.exercise.EvaluationCriterionResult;
import com.sqlteacher.application.exercise.ExerciseEvaluationResult;
import com.sqlteacher.application.exercise.SqlExerciseEvaluationService;
import com.sqlteacher.application.risk.SqlRiskAnalysis;
import com.sqlteacher.application.risk.SqlRiskAnalysisService;
import com.sqlteacher.domain.exercise.ExerciseDataset;
import com.sqlteacher.domain.exercise.ExerciseDefinition;
import com.sqlteacher.domain.exercise.ExerciseEvaluationRule;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.io.IOException;
import java.math.BigDecimal;
import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.sql.Statement;
import java.time.Duration;
import java.util.ArrayList;
import java.util.Base64;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
```

```
import java.util.regex.Pattern;

public final class DeterministicSqlExerciseEvaluationService implements SqlExerciseEvaluationService {
    private static final Logger log = LoggerFactory.getLogger(DeterministicSqlExerciseEvaluationService.class);
    private static final int MAX_EVALUATION_ROWS = 5_000;
    private static final int QUERY_TIMEOUT_SECONDS = 10;

    private final SqlRiskAnalysisService riskAnalysisService;
    private final Path evaluationDirectory;

    public DeterministicSqlExerciseEvaluationService(
        SqlRiskAnalysisService riskAnalysisService,
        SqlTeacherConfiguration configuration
    ) {
        this.riskAnalysisService = riskAnalysisService;
        this.evaluationDirectory = configuration.dataDirectory().resolve("exercise-evaluation").toAbsolutePath().normalize();
    }

    @Override
    public ExerciseEvaluationResult evaluate(
        ExerciseDefinition exercise,
        ExerciseDataset dataset,
        String submittedSql
    ) {
        long started = System.nanoTime();
        SqlRiskAnalysis risk = riskAnalysisService.analyze(submittedSql, DatabaseDialect.SQLITE);
        if (!risk.executable() || risk.multiStatement() || !"SELECT".equals(risk.statementType())) {
            return failure(
                started,
                "SQL_SAFETY_REJECTED",
                new EvaluationCriterionResult("safety", false, "只允许提交单条只读 SELECT 查询。")
            );
        }

        Path databasePath = null;
        try {
            Files.createDirectories(evaluationDirectory);
            databasePath = Files.createTempFile(evaluationDirectory, "evaluation-", ".db");
            initializeDataset(databasePath, dataset);
            QueryResult expected = executeQuery(databasePath, exercise.referenceSql());
            QueryResult actual = executeQuery(databasePath, submittedSql);
            if (expected.truncated() || actual.truncated()) {
                return failure(
                    started,
                    "RESULT_LIMIT_EXCEEDED",
                    new EvaluationCriterionResult("result_limit", false, "结果规模超过评测上限，请缩小查询范围。")
                );
            }
            List<EvaluationCriterionResult> criteria = evaluateCriteria(
                exercise.evaluationRule(), submittedSql, expected, actual
```

```
            );
            boolean passed = criteria.stream().allMatch(EvaluationCriterionResult::passed);
            return new ExerciseEvaluationResult(
                passed,
                criteria,
                passed ? "提交通过，结果满足题目要求。" : "提交未通过，请根据分项反馈调整查询。",
                elapsed(started),
                passed ? "" : "RESULT_MISMATCH"
            );
        } catch (SQLException error) {
            return failure(
                started,
                "SQL_EXECUTION_FAILED",
                new EvaluationCriterionResult("execution", false, "SQL 未能在评测数据集上执行，请检查语法和字段。")
            );
        } catch (IOException error) {
            return failure(
                started,
                "EVALUATION_ENVIRONMENT_FAILED",
                new EvaluationCriterionResult("environment", false, "暂时无法创建评测环境，请稍后重试。")
            );
        } finally {
            deleteQuietly(databasePath);
        }
    }

    private static List<EvaluationCriterionResult> evaluateCriteria(
        ExerciseEvaluationRule rule,
        String submittedSql,
        QueryResult expected,
        QueryResult actual
    ) {
        List<EvaluationCriterionResult> criteria = new ArrayList<>();
        if (rule.compareColumns()) {
            boolean passed = normalizeColumns(expected.columns()).equals(normalizeColumns(actual.columns()));
            criteria.add(new EvaluationCriterionResult(
                "columns", passed, passed ? "结果列满足要求。" : "结果列的数量、名称或顺序不符合要求。"
            ));
        }
        if (rule.compareRows()) {
            boolean passed = rowMultiset(expected.rows()).equals(rowMultiset(actual.rows()));
            criteria.add(new EvaluationCriterionResult(
                "rows", passed, passed ? "结果行和值满足要求。" : "结果行数或部分值不符合要求。"
            ));
            if (rule.rowOrderMatters()) {
                boolean ordered = expected.rows().equals(actual.rows());
                criteria.add(new EvaluationCriterionResult(
                    "order", ordered, ordered ? "结果顺序满足要求。" : "结果内容接近，但行顺序不符合要求。"
                ));
            }
```

```
    }
    if (rule.expectedRowCount() != null) {
        boolean passed = actual.rows().size() == rule.expectedRowCount();
        criteria.add(new EvaluationCriterionResult(
            "row_count", passed, passed ? "结果行数满足要求。" : "结果行数不符合题目要求。"
        ));
    }
    if (!rule.requiredSqlKeywords().isEmpty()) {
        List<String> missing = rule.requiredSqlKeywords().stream()
            .filter(keyword -> !containsSqlStructure(submittedSql, keyword))
            .toList();
        criteria.add(new EvaluationCriterionResult(
            "structure",
            missing.isEmpty(),
            missing.isEmpty() ? "SQL 结构满足要求。" : "查询尚未使用题目要求的 SQL 结构。"
        ));
    }
    return List.copyOf(criteria);
}

private static void initializeDataset(Path databasePath, ExerciseDataset dataset) throws SQLException {
    SqliteDriver.ensureLoaded();
    try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + databasePath)) {
        connection.setAutoCommit(false);
        try (Statement statement = connection.createStatement()) {
            for (String sql : SqlScriptSplitter.split(dataset.setupSql())) {
                statement.execute(sql);
            }
            connection.commit();
        } catch (SQLException | RuntimeException error) {
            connection.rollback();
            throw error;
        }
    }
}

private static QueryResult executeQuery(Path databasePath, String sql) throws SQLException {
    try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + databasePath);
         Statement settings = connection.createStatement()) {
        settings.execute("pragma query_only = on");
        try (Statement statement = connection.createStatement()) {
            statement.setQueryTimeout(QUERY_TIMEOUT_SECONDS);
            statement.setMaxRows(MAX_EVALUATION_ROWS + 1);
            try (ResultSet result = statement.executeQuery(sql)) {
                ResultSetMetaData metadata = result.getMetaData();
                int columnCount = metadata.getColumnCount();
                List<String> columns = new ArrayList<>(columnCount);
                for (int index = 1; index <= columnCount; index++) {
                    columns.add(metadata.getColumnLabel(index));
                }
```

```
                List<List<Object>> rows = new ArrayList<>();
                boolean truncated = false;
                while (result.next()) {
                    if (rows.size() >= MAX_EVALUATION_ROWS) {
                        truncated = true;
                        break;
                    }
                    List<Object> row = new ArrayList<>(columnCount);
                    for (int index = 1; index <= columnCount; index++) {
                        row.add(normalizeValue(result.getObject(index)));
                    }
                    rows.add(Collections.unmodifiableList(new ArrayList<>(row)));
                }
                return new QueryResult(List.copyOf(columns), List.copyOf(rows), truncated);
            }
        }
    }
}

private static List<String> normalizeColumns(List<String> columns) {
    return columns.stream().map(column -> column.trim().toLowerCase(Locale.ROOT)).toList();
}

private static Map<List<Object>, Long> rowMultiset(List<List<Object>> rows) {
    Map<List<Object>, Long> counts = new LinkedHashMap<>();
    for (List<Object> row : rows) {
        counts.merge(row, 1L, Long::sum);
    }
    return counts;
}

private static Object normalizeValue(Object value) {
    if (value instanceof Number number) {
        try {
            return new BigDecimal(number.toString()).stripTrailingZeros();
        } catch (NumberFormatException ignored) {
            return number.toString();
        }
    }
    if (value instanceof byte[] bytes) {
        return Base64.getEncoder().encodeToString(bytes);
    }
    return value;
}

private static boolean containsSqlStructure(String sql, String keyword) {
    String normalized = maskCommentsAndLiterals(sql).toUpperCase(Locale.ROOT).replaceAll("\\s+", " ").trim();
    String phrase = keyword.toUpperCase(Locale.ROOT).trim().replaceAll("\\s+", " ");
    return Pattern.compile("(?<![A-Z0-9_])" + Pattern.quote(phrase) + "(?![A-Z0-9_])")
        .matcher(normalized)
```

```
        .find();
}

private static String maskCommentsAndLiterals(String sql) {
    StringBuilder result = new StringBuilder(sql.length());
    char quote = '\0';
    boolean lineComment = false;
    boolean blockComment = false;
    for (int index = 0; index < sql.length(); index++) {
        char current = sql.charAt(index);
        char next = index + 1 < sql.length() ? sql.charAt(index + 1) : '\0';
        if (lineComment) {
            if (current == '\n' || current == '\r') {
                lineComment = false;
                result.append(' ');
            }
            continue;
        }
        if (blockComment) {
            if (current == '*' && next == '/') {
                index++;
                blockComment = false;
                result.append(' ');
            }
            continue;
        }
        if (quote == '\0' && current == '-' && next == '-') {
            index++;
            lineComment = true;
            result.append(' ');
        } else if (quote == '\0' && current == '/' && next == '*') {
            index++;
            blockComment = true;
            result.append(' ');
        } else if (quote == '\0' && (current == '\'' || current == '"' || current == '`')) {
            quote = current;
            result.append(' ');
        } else if (quote != '\0') {
            result.append(' ');
            if (current == quote && next == quote) {
                result.append(' ');
                index++;
            } else if (current == quote) {
                quote = '\0';
            }
        } else {
            result.append(current);
        }
    }
    return result.toString();
```

```
    }

    private static ExerciseEvaluationResult failure(
        long started,
        String errorCode,
        EvaluationCriterionResult criterion
    ) {
        return new ExerciseEvaluationResult(
            false, List.of(criterion), "提交未通过，请根据反馈修改后重试。", elapsed(started), errorCode
        );
    }

    private static Duration elapsed(long started) {
        return Duration.ofNanos(System.nanoTime() - started);
    }

    private static void deleteQuietly(Path databasePath) {
        if (databasePath == null) {
            return;
        }
        try {
            Files.deleteIfExists(databasePath);
            Files.deleteIfExists(Path.of(databasePath + "-wal"));
            Files.deleteIfExists(Path.of(databasePath + "-shm"));
        } catch (IOException error) {
            log.warn("Failed to remove temporary exercise evaluation database", error);
        }
    }

    private record QueryResult(List<String> columns, List<List<Object>> rows, boolean truncated) {
    }
}
// FILE: src/main/java/com/sqlteacher/infrastructure/database/JdbcExercisePracticeService.java
package com.sqlteacher.infrastructure.database;

import com.sqlteacher.application.config.SqlTeacherConfiguration;
import com.sqlteacher.application.connection.DatabaseDialect;
import com.sqlteacher.application.execution.SqlExecutionResult;
import com.sqlteacher.application.exercise.EvaluationCriterionResult;
import com.sqlteacher.application.exercise.ExerciseAttemptResult;
import com.sqlteacher.application.exercise.ExerciseEvaluationResult;
import com.sqlteacher.application.exercise.ExerciseHint;
import com.sqlteacher.application.exercise.ExerciseManagementService;
import com.sqlteacher.application.exercise.ExercisePracticeService;
import com.sqlteacher.application.exercise.ExerciseSession;
import com.sqlteacher.application.exercise.ExerciseView;
import com.sqlteacher.application.exercise.SqlExerciseEvaluationService;
import com.sqlteacher.application.event.LearningEventService;
import com.sqlteacher.application.risk.SqlRiskAnalysis;
import com.sqlteacher.application.risk.SqlRiskAnalysisService;
```

```
import com.sqlteacher.domain.SqlTeacherException;
import com.sqlteacher.domain.exercise.ExerciseAttemptStatus;
import com.sqlteacher.domain.exercise.ExerciseDataset;
import com.sqlteacher.domain.exercise.ExerciseDefinition;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.Types;
import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.UUID;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public final class JdbcExercisePracticeService implements ExercisePracticeService {
    private static final Logger log = LoggerFactory.getLogger(JdbcExercisePracticeService.class);
    static final int MAX_RESULT_ROWS = 500;
    private static final int QUERY_TIMEOUT_SECONDS = 10;

    private final JdbcConnectionFactory connectionFactory;
    private final ExerciseManagementService managementService;
    private final SqlRiskAnalysisService riskAnalysisService;
    private final SqlExerciseEvaluationService evaluationService;
    private final SqlResultMapper resultMapper;
    private final ExerciseAttemptCodec attemptCodec;
    private final LearningEventService learningEventService;
    private final Path sessionDirectory;

    public JdbcExercisePracticeService(
        JdbcConnectionFactory connectionFactory,
        ExerciseManagementService managementService,
        SqlRiskAnalysisService riskAnalysisService,
        SqlExerciseEvaluationService evaluationService,
        SqlResultMapper resultMapper,
        SqlTeacherConfiguration configuration,
        LearningEventService learningEventService
    ) {
        this.connectionFactory = connectionFactory;
        this.managementService = managementService;
        this.riskAnalysisService = riskAnalysisService;
```

```
    this.evaluationService = evaluationService;
    this.resultMapper = resultMapper;
    this.attemptCodec = new ExerciseAttemptCodec();
    this.learningEventService = learningEventService;
    this.sessionDirectory = configuration.dataDirectory().resolve("exercise-sessions").toAbsolutePath().normalize();
}

@Override
public ExerciseSession start(String exerciseId) {
    ExerciseDefinition exercise = requireAvailableExercise(exerciseId);
    ExerciseDataset dataset = requireDataset(exercise.datasetId());
    String sessionId = UUID.randomUUID().toString();
    Instant startedAt = Instant.now();
    Path databasePath = sessionDatabase(sessionId);
    try {
        initializeDataset(databasePath, dataset);
        try (Connection connection = connectionFactory.open("app");
             PreparedStatement statement = connection.prepareStatement("""
                 insert into exercise_sessions(id, exercise_id, exercise_version, started_at, hints_used)
                 values (?, ?, ?, ?, 0)
                 """)) {
            statement.setString(1, sessionId);
            statement.setString(2, exercise.id());
            statement.setInt(3, exercise.version());
            statement.setString(4, startedAt.toString());
            statement.executeUpdate();
        }
        return toSession(sessionId, exercise, startedAt, 0, false);
    } catch (SQLException | IOException error) {
        deleteSessionDatabase(databasePath);
        throw new SqlTeacherException("EXERCISE_SESSION_START_FAILED", "Failed to start exercise session", error);
    }
}

@Override
public ExerciseSession reset(String sessionId) {
    SessionRecord session = requireSession(sessionId, true);
    ExerciseDefinition exercise = requireCurrentExercise(session);
    ExerciseDataset dataset = requireDataset(exercise.datasetId());
    Path databasePath = sessionDatabase(session.id());
    deleteSessionDatabase(databasePath);
    try {
        initializeDataset(databasePath, dataset);
        return toSession(session.id(), exercise, session.startedAt(), session.hintsUsed(), false);
    } catch (SQLException | IOException error) {
        deleteSessionDatabase(databasePath);
        throw new SqlTeacherException("EXERCISE_SESSION_RESET_FAILED", "Failed to reset exercise dataset", error);
    }
}
```

```
@Override
public ExerciseAttemptResult run(String sessionId, String sql) {
    SessionRecord session = requireSession(sessionId, true);
    requireCurrentExercise(session);
    SqlExecutionResult execution = executeStudentQuery(session.id(), sql);
    Instant occurredAt = Instant.now();
    String attemptId = UUID.randomUUID().toString();
    recordAttempt(
        attemptId, session.id(), ExerciseAttemptStatus.RUN, sql, execution, null, occurredAt,
        execution.success() ? "" : "SQL_EXECUTION_FAILED"
    );
    learningEventService.recordExerciseAttempt(
        session.exerciseId(), ExerciseAttemptStatus.RUN.name(), execution.success(), execution.duration(),
        execution.success() ? null : "SQL_EXECUTION_FAILED"
    );
    return new ExerciseAttemptResult(
        attemptId, session.id(), ExerciseAttemptStatus.RUN, execution, null, occurredAt
    );
}

@Override
public ExerciseAttemptResult submit(String sessionId, String sql) {
    SessionRecord session = requireSession(sessionId, true);
    ExerciseDefinition exercise = requireCurrentExercise(session);
    ExerciseDataset dataset = requireDataset(exercise.datasetId());
    SqlExecutionResult execution = executeStudentQuery(session.id(), sql);
    ExerciseEvaluationResult evaluation = execution.success()
        ? evaluationService.evaluate(exercise, dataset, sql)
        : failedExecutionEvaluation(execution.duration());
    ExerciseAttemptStatus status = evaluation.passed()
        ? ExerciseAttemptStatus.PASSED
        : ExerciseAttemptStatus.FAILED;
    Instant occurredAt = Instant.now();
    String attemptId = UUID.randomUUID().toString();
    recordAttempt(attemptId, session.id(), status, sql, execution, evaluation, occurredAt, evaluation.errorCode());
    learningEventService.recordExerciseAttempt(
        exercise.id(), status.name(), evaluation.passed(),
        execution.duration().plus(evaluation.duration()), evaluation.errorCode()
    );
    if (evaluation.passed()) {
        completeSession(session.id(), occurredAt);
        deleteSessionDatabase(sessionDatabase(session.id()));
    }
    return new ExerciseAttemptResult(attemptId, session.id(), status, execution, evaluation, occurredAt);
}

@Override
public ExerciseHint requestHint(String sessionId) {
    SessionRecord session = requireSession(sessionId, true);
    ExerciseDefinition exercise = requireCurrentExercise(session);
```

```
    if (exercise.hints().isEmpty()) {
        throw new SqlTeacherException("EXERCISE_HINT_UNAVAILABLE", "No hint is available for this exercise");
    }
    int nextCount = Math.min(session.hintsUsed() + 1, exercise.hints().size());
    if (nextCount > session.hintsUsed()) {
        try (Connection connection = connectionFactory.open("app");
             PreparedStatement statement = connection.prepareStatement(
                 "update exercise_sessions set hints_used = ? where id = ? and completed_at is null"
             )) {
            statement.setInt(1, nextCount);
            statement.setString(2, session.id());
            if (statement.executeUpdate() != 1) {
                throw sessionClosed();
            }
        } catch (SQLException error) {
            throw new SqlTeacherException("EXERCISE_HINT_FAILED", "Failed to record exercise hint", error);
        }
        learningEventService.recordExerciseHint(exercise.id(), nextCount);
    }
    return new ExerciseHint(
        nextCount,
        exercise.hints().get(nextCount - 1),
        nextCount >= exercise.hints().size()
    );
}

@Override
public void close(String sessionId) {
    SessionRecord session = requireSession(sessionId, false);
    if (!session.completed()) {
        completeSession(session.id(), Instant.now());
    }
    deleteSessionDatabase(sessionDatabase(session.id()));
}

public void shutdown() {
    try (Connection connection = connectionFactory.open("app")) {
        ExerciseSessionRuntimeCleaner.closeActiveSessions(connection, Instant.now());
        ExerciseSessionRuntimeCleaner.deleteSessionFiles(sessionDirectory);
    } catch (SQLException | IOException error) {
        log.warn("Failed to clean exercise sessions during application shutdown", error);
    }
}

private SqlExecutionResult executeStudentQuery(String sessionId, String sql) {
    SqlRiskAnalysis risk = riskAnalysisService.analyze(sql, DatabaseDialect.SQLITE);
    if (!risk.executable() || risk.multiStatement() || !"SELECT".equals(risk.statementType())) {
        return new SqlExecutionResult(
            false, List.of(), List.of(), 0, false,
            "练习环境只允许执行单条 SELECT 查询。", Duration.ZERO
```

```
        );
    }
    long started = System.nanoTime();
    try {
        SqliteDriver.ensureLoaded();
        try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + sessionDatabase(sessionId));
             Statement settings = connection.createStatement()) {
            settings.execute("pragma query_only = on");
            try (PreparedStatement statement = connection.prepareStatement(sql)) {
                statement.setQueryTimeout(QUERY_TIMEOUT_SECONDS);
                statement.setMaxRows(MAX_RESULT_ROWS + 1);
                try (ResultSet rows = statement.executeQuery()) {
                    return resultMapper.mapQueryResult(
                        rows, Duration.ofNanos(System.nanoTime() - started), MAX_RESULT_ROWS
                    );
                }
            }
        }
    } catch (SQLException error) {
        return new SqlExecutionResult(
            false, List.of(), List.of(), 0, false,
            "SQL 执行失败，请检查语法、表名和字段名。",
            Duration.ofNanos(System.nanoTime() - started)
        );
    }
}

private void recordAttempt(
    String attemptId,
    String sessionId,
    ExerciseAttemptStatus status,
    String sql,
    SqlExecutionResult execution,
    ExerciseEvaluationResult evaluation,
    Instant occurredAt,
    String errorCode
) {
    String insert = """
        insert into exercise_attempts(
            id, session_id, status, sql_text, execution_success, passed, duration_ms,
            result_columns_json, result_rows_json, feedback_json, error_code, created_at
        ) values (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """;
    try (Connection connection = connectionFactory.open("app");
         PreparedStatement statement = connection.prepareStatement(insert)) {
        statement.setString(1, attemptId);
        statement.setString(2, sessionId);
        statement.setString(3, status.name());
        statement.setString(4, sql == null ? "" : sql);
        statement.setBoolean(5, execution.success());
```

```
        if (evaluation == null) {
            statement.setNull(6, Types.INTEGER);
        } else {
            statement.setBoolean(6, evaluation.passed());
        }
        statement.setLong(7, durationMillis(execution, evaluation));
        statement.setString(8, attemptCodec.encode(execution.columns()));
        statement.setString(9, attemptCodec.encode(execution.rows()));
        statement.setString(10, attemptCodec.encode(feedback(evaluation)));
        statement.setString(11, errorCode == null || errorCode.isBlank() ? null : errorCode);
        statement.setString(12, occurredAt.toString());
        statement.executeUpdate();
    } catch (SQLException error) {
        throw new SqlTeacherException("EXERCISE_ATTEMPT_RECORD_FAILED", "Failed to record exercise attempt", error);
    }
}

private static long durationMillis(SqlExecutionResult execution, ExerciseEvaluationResult evaluation) {
    Duration duration = execution.duration();
    if (evaluation != null) {
        duration = duration.plus(evaluation.duration());
    }
    return Math.max(0, duration.toMillis());
}

private static List<String> feedback(ExerciseEvaluationResult evaluation) {
    if (evaluation == null) {
        return List.of();
    }
    List<String> messages = new ArrayList<>();
    if (!evaluation.feedback().isBlank()) {
        messages.add(evaluation.feedback());
    }
    messages.addAll(evaluation.criteria().stream().map(EvaluationCriterionResult::feedback).toList());
    return List.copyOf(messages);
}

private static ExerciseEvaluationResult failedExecutionEvaluation(Duration duration) {
    return new ExerciseEvaluationResult(
        false,
        List.of(new EvaluationCriterionResult("execution", false, "SQL 未能成功执行，请先修正语法或字段。")),
        "本次提交未通过：SQL 执行失败。",
        duration,
        "SQL_EXECUTION_FAILED"
    );
}

private SessionRecord requireSession(String sessionId, boolean active) {
    String normalizedId = validateSessionId(sessionId);
    String sql = "select id, exercise_id, exercise_version, started_at, hints_used, completed_at from exercise_sessions where id = ?";
```

```
    try (Connection connection = connectionFactory.open("app");
         PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setString(1, normalizedId);
        try (ResultSet row = statement.executeQuery()) {
            if (!row.next()) {
                throw new SqlTeacherException("EXERCISE_SESSION_NOT_FOUND", "Exercise session not found");
            }
            SessionRecord session = new SessionRecord(
                row.getString("id"), row.getString("exercise_id"), row.getInt("exercise_version"),
                Instant.parse(row.getString("started_at")), row.getInt("hints_used"),
                row.getString("completed_at") != null
            );
            if (active && session.completed()) {
                throw sessionClosed();
            }
            return session;
        }
    } catch (SQLException error) {
        throw new SqlTeacherException("EXERCISE_SESSION_READ_FAILED", "Failed to read exercise session", error);
    }
}

private ExerciseDefinition requireCurrentExercise(SessionRecord session) {
    ExerciseDefinition exercise = managementService.findDefinition(session.exerciseId())
        .orElseThrow(() -> new SqlTeacherException("EXERCISE_NOT_FOUND", "Exercise not found"));
    if (exercise.version() != session.exerciseVersion()) {
        throw new SqlTeacherException(
            "EXERCISE_SESSION_STALE", "The exercise changed after this session started; start a new session"
        );
    }
    return exercise;
}

private ExerciseDefinition requireAvailableExercise(String exerciseId) {
    ExerciseDefinition exercise = managementService.findDefinition(exerciseId)
        .orElseThrow(() -> new SqlTeacherException("EXERCISE_NOT_FOUND", "Exercise not found"));
    if (!exercise.enabled()) {
        throw new SqlTeacherException("EXERCISE_DISABLED", "Exercise is not available");
    }
    return exercise;
}

private ExerciseDataset requireDataset(String datasetId) {
    return managementService.listDatasets().stream()
        .filter(dataset -> dataset.id().equals(datasetId))
        .findFirst()
        .orElseThrow(() -> new SqlTeacherException("EXERCISE_DATASET_NOT_FOUND", "Exercise dataset not found"));
}

private void initializeDataset(Path databasePath, ExerciseDataset dataset) throws SQLException, IOException {
```

```
    Files.createDirectories(sessionDirectory);
    SqliteDriver.ensureLoaded();
    try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + databasePath)) {
        connection.setAutoCommit(false);
        try (Statement statement = connection.createStatement()) {
            for (String sql : SqlScriptSplitter.split(dataset.setupSql())) {
                statement.execute(sql);
            }
            connection.commit();
        } catch (SQLException | RuntimeException error) {
            connection.rollback();
            throw error;
        }
    }
}

private void completeSession(String sessionId, Instant completedAt) {
    try (Connection connection = connectionFactory.open("app");
         PreparedStatement statement = connection.prepareStatement(
             "update exercise_sessions set completed_at = ? where id = ? and completed_at is null"
         )) {
        statement.setString(1, completedAt.toString());
        statement.setString(2, sessionId);
        statement.executeUpdate();
    } catch (SQLException error) {
        throw new SqlTeacherException("EXERCISE_SESSION_CLOSE_FAILED", "Failed to close exercise session", error);
    }
}

private Path sessionDatabase(String sessionId) {
    String normalizedId = validateSessionId(sessionId);
    Path path = sessionDirectory.resolve(normalizedId + ".db").normalize();
    if (!path.getParent().equals(sessionDirectory)) {
        throw new IllegalArgumentException("Invalid session ID");
    }
    return path;
}

private static String validateSessionId(String sessionId) {
    if (sessionId == null) {
        throw new IllegalArgumentException("sessionId must not be null");
    }
    return UUID.fromString(sessionId.trim()).toString();
}

private static void deleteSessionDatabase(Path databasePath) {
    try {
        Files.deleteIfExists(databasePath);
        Files.deleteIfExists(Path.of(databasePath + "-wal"));
        Files.deleteIfExists(Path.of(databasePath + "-shm"));
```

```
        } catch (IOException error) {
            throw new SqlTeacherException("EXERCISE_SESSION_CLEANUP_FAILED", "Failed to clean exercise dataset", error);
        }
    }

    private static ExerciseSession toSession(
        String id, ExerciseDefinition exercise, Instant startedAt, int hintsUsed, boolean completed
    ) {
        return new ExerciseSession(
            id,
            new ExerciseView(
                exercise.id(), exercise.title(), exercise.description(), exercise.knowledgePoint(),
                exercise.difficulty(), exercise.version()
            ),
            startedAt,
            hintsUsed,
            completed
        );
    }

    private static SqlTeacherException sessionClosed() {
        return new SqlTeacherException("EXERCISE_SESSION_CLOSED", "Exercise session is already closed");
    }

    private record SessionRecord(
        String id,
        String exerciseId,
        int exerciseVersion,
        Instant startedAt,
        int hintsUsed,
        boolean completed
    ) {
    }
}
// FILE: src/main/java/com/sqlteacher/infrastructure/ai/Nl2SqlServiceImpl.java
package com.sqlteacher.infrastructure.ai;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.sqlteacher.application.ai.AiCompletionRequest;
import com.sqlteacher.application.ai.AiCompletionResult;
import com.sqlteacher.application.ai.AiModelProvider;
import com.sqlteacher.application.ai.AiModelSelection;
import com.sqlteacher.application.ai.AiModelSelectionService;
import com.sqlteacher.application.config.AiConfiguration;
import com.sqlteacher.application.connection.ConnectionManagementService;
import com.sqlteacher.application.connection.DatabaseDialect;
import com.sqlteacher.application.event.LearningEventService;
import com.sqlteacher.application.metadata.DatabaseColumn;
import com.sqlteacher.application.metadata.DatabaseMetadataService;
import com.sqlteacher.application.metadata.DatabaseTable;
```

```
import com.sqlteacher.application.nl2sql.Nl2SqlPlan;
import com.sqlteacher.application.nl2sql.Nl2SqlRequest;
import com.sqlteacher.application.nl2sql.Nl2SqlService;
import com.sqlteacher.application.nl2sql.SqlErrorExplanation;
import com.sqlteacher.infrastructure.ai.dto.OllamaNl2SqlResponse;
import com.sqlteacher.infrastructure.ai.dto.OllamaSqlErrorResponse;
import com.sqlteacher.domain.SqlTeacherException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.List;
import java.util.Objects;

public final class Nl2SqlServiceImpl implements Nl2SqlService {
    private static final Logger log = LoggerFactory.getLogger(Nl2SqlServiceImpl.class);
    private static final String PROMPT_VERSION = "v3";
    private final AiModelProvider aiModelProvider;
    private final AiConfiguration aiConfiguration;
    private final AiModelSelectionService modelSelectionService;
    private final DatabaseMetadataService databaseMetadataService;
    private final LearningEventService learningEventService;
    private final ObjectMapper objectMapper;
    private final ConnectionManagementService connectionManagementService;

    public Nl2SqlServiceImpl(
        AiModelProvider aiModelProvider,
        AiConfiguration aiConfiguration,
        DatabaseMetadataService databaseMetadataService,
        LearningEventService learningEventService
    ) {
        this(
            aiModelProvider,
            aiConfiguration,
            AiModelSelectionService.fixed(aiConfiguration.defaultModel()),
            databaseMetadataService,
            learningEventService,
            null
        );
    }

    public Nl2SqlServiceImpl(
        AiModelProvider aiModelProvider,
        AiConfiguration aiConfiguration,
        AiModelSelectionService modelSelectionService,
        DatabaseMetadataService databaseMetadataService,
        LearningEventService learningEventService
    ) {
        this(
            aiModelProvider,
            aiConfiguration,
```

```
                "",
                "",
                aiResult.errorMessage(),
                aiResult.model(),
                PROMPT_VERSION
            );
        }

        try {
            OllamaNl2SqlResponse response = objectMapper.readValue(aiResult.content(), OllamaNl2SqlResponse.class);

            String validationError = validateAiResponse(response);
            if (validationError != null) {
                log.warn("AI response validation failed: {}", validationError);
                recordAiGeneration(request.connectionId(), false, aiResult.model(), "VALIDATION_FAILED");
                return new Nl2SqlPlan(
                    "",
                    "",
                    validationError,
                    aiResult.model(),
                    PROMPT_VERSION
                );
            }

            recordAiGeneration(request.connectionId(), true, aiResult.model(), null);
            return new Nl2SqlPlan(
                response.sqlDraft(),
                response.intent(),
                response.explanation(),
                aiResult.model(),
                PROMPT_VERSION
            );
        } catch (Exception ex) {
            log.warn("Failed to parse AI response", ex);
            recordAiGeneration(request.connectionId(), false, aiResult.model(), "PARSE_ERROR");
            return new Nl2SqlPlan(
                "",
                "",
                "Failed to parse AI response: " + ex.getClass().getSimpleName(),
                aiResult.model(),
                PROMPT_VERSION
            );
        }
    }

    private void recordAiGeneration(String connectionId, boolean successful, String model, String errorCode) {
        try {
            learningEventService.recordAiGeneration(connectionId, successful, model, PROMPT_VERSION, errorCode);
        } catch (Exception ex) {
            log.warn("Failed to record AI generation event", ex);
```

```
    }
}

private String validateAiResponse(OllamaNl2SqlResponse response) {
    if (response.sqlDraft() == null || response.sqlDraft().isBlank()) {
        return "AI generated empty SQL draft";
    }

    if (!"QUERY".equalsIgnoreCase(response.intent())) {
        return "AI generated invalid intent: " + response.intent();
    }

    if (response.explanation() == null || response.explanation().isBlank()) {
        return "AI generated empty explanation";
    }

    return null;
}

private String buildPrompt(String naturalLanguage, String connectionId) {
    StringBuilder sb = new StringBuilder();
    String databaseName = databaseDialect(connectionId).name();
    sb.append("You are a SQL teacher assistant. Convert the following natural language query into a valid ")
        .append(databaseName).append(" SELECT statement.\n");
    sb.append("\n");
    sb.append("Database: ").append(databaseName).append("\n");
    sb.append("Available tables:\n");
    sb.append(buildTableSchema(connectionId));
    sb.append("\n");
    sb.append("IMPORTANT RULES:\n");
    sb.append("- ONLY generate SELECT statements\n");
    sb.append("- NO INSERT, UPDATE, DELETE, CREATE, DROP, ALTER, or any modifying statements\n");
    sb.append("- NO multiple statements separated by semicolons\n");
    sb.append("- Limit the result to at most 500 rows using a LIMIT clause\n");
    sb.append("- If the user requests a smaller number of rows, preserve that smaller LIMIT\n");
    sb.append("- Return ONLY a valid JSON object\n");
    sb.append("\n");
    sb.append("Natural language: ").append(naturalLanguage).append("\n");
    sb.append("\n");
    sb.append("Return ONLY a valid JSON object with these fields:\n");
    sb.append("- sqlDraft: the generated SELECT SQL statement\n");
    sb.append("- intent: must be \"QUERY\"\n");
    sb.append("- explanation: detailed teaching explanation including:\n");
    sb.append("  1. What the query does (purpose)\n");
    sb.append("  2. Which tables are involved\n");
    sb.append("  3. What conditions are used and why\n");
    sb.append("  4. What columns are selected\n");
    sb.append("  5. Expected result format\n");
    sb.append("\n");
    sb.append("Example output:\n");
```

```
        sb.append("{\"sqlDraft\": \"SELECT name, score FROM student WHERE score >= 60 LIMIT 500\", \"intent\": \"QUERY\", \"explanation\": \"该查询从 student 表中选取成绩大于等于 60 的学生记录，返回姓名和分数两列，限制最多 500 条结果。WHERE 子句用于过滤符合条件的行。\"}");
        return sb.toString();
    }

    private String buildTableSchema(String connectionId) {
        try {
            List<DatabaseTable> tables = databaseMetadataService.listTables(connectionId);
            if (tables == null || tables.isEmpty()) {
                return getDefaultTableSchema();
            }
            StringBuilder sb = new StringBuilder();
            for (DatabaseTable table : tables) {
                sb.append("  - ").append(table.name());
                sb.append(" (");
                List<String> columnNames = table.columns().stream()
                    .map(DatabaseColumn::name)
                    .toList();
                sb.append(String.join(", ", columnNames));
                sb.append(")\n");
            }
            return sb.toString().trim();
        } catch (SqlTeacherException error) {
            throw error;
        } catch (Exception ex) {
            return getDefaultTableSchema();
        }
    }

    private DatabaseDialect databaseDialect(String connectionId) {
        if (connectionManagementService == null) {
            return DatabaseDialect.SQLITE;
        }
        return connectionManagementService.findProfile(connectionId)
            .map(profile -> profile.dialect())
            .orElseThrow(() -> new SqlTeacherException(
                "DATABASE_CONNECTION_NOT_FOUND",
                "找不到所选数据库连接，请在设置页重新选择。"
            ));
    }

    private String getDefaultTableSchema() {
        return "  - student (id, name, score, class_id)\n  - class (id, name, teacher)";
    }

    @Override
    public SqlErrorExplanation explainSqlError(String connectionId, String sql, String errorMessage) {
        Objects.requireNonNull(connectionId, "connectionId must not be null");
        Objects.requireNonNull(sql, "sql must not be null");
        Objects.requireNonNull(errorMessage, "errorMessage must not be null");
        if (connectionId.isBlank()) {
```

```
    throw new IllegalArgumentException("connectionId must not be blank");
}
if (sql.isBlank()) {
    throw new IllegalArgumentException("sql must not be blank");
}
if (errorMessage.isBlank()) {
    throw new IllegalArgumentException("errorMessage must not be blank");
}

String prompt = buildErrorExplanationPrompt(sql, errorMessage, connectionId);
String selectedModel = resolveSelectedModel();
if (selectedModel.isEmpty()) {
    return SqlErrorExplanation.failure(
        "No local Ollama model is installed",
        aiConfiguration.defaultModel()
    );
}
AiCompletionRequest aiRequest = new AiCompletionRequest(
    selectedModel,
    prompt,
    aiConfiguration.generateTimeout()
);

AiCompletionResult aiResult = aiModelProvider.complete(aiRequest);

if (!aiResult.success()) {
    recordAiGeneration(connectionId, false, aiResult.model(), "AI_PROVIDER_FAILED");
    return SqlErrorExplanation.failure(aiResult.errorMessage(), aiResult.model());
}

try {
    OllamaSqlErrorResponse response = objectMapper.readValue(aiResult.content(), OllamaSqlErrorResponse.class);

    String validationError = validateErrorExplanationResponse(response);
    if (validationError != null) {
        log.warn("AI error explanation validation failed: {}", validationError);
        recordAiGeneration(connectionId, false, aiResult.model(), "VALIDATION_FAILED");
        return SqlErrorExplanation.failure(validationError, aiResult.model());
    }

    recordAiGeneration(connectionId, true, aiResult.model(), null);
    return SqlErrorExplanation.success(
        response.errorCause(),
        response.correctionSuggestion(),
        response.correctedSql(),
        aiResult.model()
    );
} catch (Exception ex) {
    log.warn("Failed to parse AI error explanation response", ex);
    recordAiGeneration(connectionId, false, aiResult.model(), "PARSE_ERROR");
```

```
            return SqlErrorExplanation.failure("Failed to parse AI error explanation: " + ex.getClass().getSimpleName(), aiResult.model());
        }
    }

    private String resolveSelectedModel() {
        String preferred = aiModelProvider.preferredModel();
        if (!preferred.isBlank()) {
            return preferred;
        }
        AiModelSelection selection = modelSelectionService.current();
        if (!selection.hasSelection()) {
            selection = modelSelectionService.refresh();
        }
        return selection.selectedModel();
    }

    private Nl2SqlPlan unavailableModelPlan(String connectionId) {
        String model = aiConfiguration.defaultModel();
        recordAiGeneration(connectionId, false, model, "MODEL_UNAVAILABLE");
        return new Nl2SqlPlan(
            "",
            "",
            "No local Ollama model is installed. Install a model or refresh the model list.",
            model,
            PROMPT_VERSION
        );
    }

    private String validateErrorExplanationResponse(OllamaSqlErrorResponse response) {
        if (response.errorCause().isBlank()) {
            return "AI generated empty error cause";
        }
        if (response.correctionSuggestion().isBlank()) {
            return "AI generated empty correction suggestion";
        }
        if (response.correctedSql().isBlank()) {
            return "AI generated empty corrected SQL draft";
        }
        return null;
    }

    private String buildErrorExplanationPrompt(String sql, String errorMessage, String connectionId) {
        StringBuilder sb = new StringBuilder();
        sb.append("You are a SQL teacher assistant. Explain the following SQL error and provide a corrected SQL statement.\n");
        sb.append("\n");
        sb.append("Database: ").append(databaseDialect(connectionId).name()).append("\n");
        sb.append("Available tables:\n");
        sb.append(buildTableSchema(connectionId));
        sb.append("\n");
        sb.append("SQL statement that caused the error:\n");
```

```
        sb.append(sql).append("\n");
        sb.append("\n");
        sb.append("Error message:\n");
        sb.append(errorMessage).append("\n");
        sb.append("\n");
        sb.append("Return ONLY a valid JSON object with these fields:\n");
        sb.append("- errorCause: explanation of what caused the error\n");
        sb.append("- correctionSuggestion: suggestion for fixing the error\n");
        sb.append("- correctedSql: the corrected SQL statement\n");
        sb.append("\n");
        sb.append("Example output:\n");
        sb.append("{\"errorCause\": \"Unknown column 'nam' in 'field list'\", \"correctionSuggestion\": \"The column 'nam' does not exist. Did you mean
'name'?\", \"correctedSql\": \"SELECT name FROM student LIMIT 500\"}");
        return sb.toString();
    }
}
// FILE: src/main/java/com/sqlteacher/application/nl2sql/DefaultNl2SqlSafetyService.java
package com.sqlteacher.application.nl2sql;

import com.sqlteacher.application.event.LearningEventService;
import com.sqlteacher.application.risk.SqlRiskAnalysis;
import com.sqlteacher.application.risk.SqlRiskAnalysisService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.Objects;

/**
 * Coordinates AI draft generation, Java-side risk analysis, and risk event recording.
 */
public final class DefaultNl2SqlSafetyService implements Nl2SqlSafetyService {
    private static final Logger log = LoggerFactory.getLogger(DefaultNl2SqlSafetyService.class);

    private final Nl2SqlService nl2SqlService;
    private final SqlRiskAnalysisService riskAnalysisService;
    private final LearningEventService learningEventService;

    public DefaultNl2SqlSafetyService(
        Nl2SqlService nl2SqlService,
        SqlRiskAnalysisService riskAnalysisService,
        LearningEventService learningEventService
    ) {
        this.nl2SqlService = Objects.requireNonNull(nl2SqlService, "nl2SqlService must not be null");
        this.riskAnalysisService = Objects.requireNonNull(
            riskAnalysisService,
            "riskAnalysisService must not be null"
        );
        this.learningEventService = Objects.requireNonNull(
            learningEventService,
            "learningEventService must not be null"
        );
```

```
    }

    @Override
    public Nl2SqlSafetyResult generateAndAssess(Nl2SqlRequest request) {
        Objects.requireNonNull(request, "request must not be null");

        Nl2SqlPlan plan = Objects.requireNonNull(
            nl2SqlService.generate(request),
            "nl2SqlService result must not be null"
        );
        SqlRiskAnalysis riskAnalysis = riskAnalysisService.analyze(plan.sqlDraft(), request.dialect());
        Nl2SqlSafetyResult result = new Nl2SqlSafetyResult(plan, riskAnalysis);

        if (result.draftAvailable() && !result.accepted()) {
            recordBlockedDraft(request, riskAnalysis);
        }

        return result;
    }

    private void recordBlockedDraft(Nl2SqlRequest request, SqlRiskAnalysis riskAnalysis) {
        try {
            learningEventService.recordSqlRiskBlocked(
                request.connectionId(),
                riskAnalysis.statementType(),
                riskAnalysis.level(),
                riskAnalysis.multiStatement()
            );
        } catch (RuntimeException error) {
            log.warn("Failed to record blocked AI SQL draft", error);
        }
    }
}
// FILE: src/main/java/com/sqlteacher/infrastructure/database/SqliteKnowledgeService.java
package com.sqlteacher.infrastructure.database;

import com.sqlteacher.application.event.LearningEventService;
import com.sqlteacher.application.knowledge.KnowledgeDocument;
import com.sqlteacher.application.knowledge.KnowledgeDocumentService;
import com.sqlteacher.application.knowledge.KnowledgeSearchResult;
import com.sqlteacher.application.knowledge.KnowledgeSearchService;
import com.sqlteacher.domain.SqlTeacherException;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.ByteBuffer;
import java.nio.charset.CharacterCodingException;
import java.nio.charset.CodingErrorAction;
import java.nio.file.Files;
import java.nio.file.Path;
```

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.Instant;
import java.util.ArrayList;
import java.util.HexFormat;
import java.util.List;
import java.util.Locale;
import java.util.Set;
import java.util.UUID;

public final class SqliteKnowledgeService implements KnowledgeDocumentService, KnowledgeSearchService {
    static final long MAX_DOCUMENT_BYTES = 2 * 1024 * 1024;
    static final int MAX_CHUNK_CHARACTERS = 800;
    private static final Set<String> ALLOWED_EXTENSIONS = Set.of("txt", "md", "markdown");

    private final JdbcConnectionFactory connectionFactory;
    private final LearningEventService eventService;

    public SqliteKnowledgeService(JdbcConnectionFactory connectionFactory, LearningEventService eventService) {
        this.connectionFactory = connectionFactory;
        this.eventService = eventService;
    }

    @Override
    public KnowledgeDocument importDocument(Path requestedPath) {
        Path path = validatePath(requestedPath);
        byte[] bytes;
        try {
            long size = Files.size(path);
            if (size < 1 || size > MAX_DOCUMENT_BYTES) {
                throw new IllegalArgumentException("Document must be between 1 byte and 2 MiB");
            }
            bytes = Files.readAllBytes(path);
        } catch (IOException error) {
            throw new SqlTeacherException("KNOWLEDGE_DOCUMENT_READ_FAILED", "Failed to read knowledge document", error);
        }
        String content = decodeUtf8(bytes).replace("\u0000", "").trim();
        if (content.isBlank()) {
            throw new IllegalArgumentException("Knowledge document must contain UTF-8 text");
        }
        List<String> chunks = chunk(content);
        String id = UUID.randomUUID().toString();
        String title = title(path, content);
        String sourceName = path.getFileName().toString();
        String hash = sha256(bytes);
        Instant importedAt = Instant.now();
```

```
    try (Connection connection = connectionFactory.open("app")) {
        connection.setAutoCommit(false);
        try {
            insertDocument(connection, id, title, sourceName, hash, chunks.size(), importedAt);
            insertChunks(connection, id, chunks);
            connection.commit();
            return new KnowledgeDocument(id, title, sourceName, chunks.size(), importedAt);
        } catch (SQLException | RuntimeException error) {
            connection.rollback();
            throw error;
        }
    } catch (SQLException error) {
        if (error.getMessage() != null && error.getMessage().contains("knowledge_documents.content_hash")) {
            throw new SqlTeacherException("KNOWLEDGE_DOCUMENT_DUPLICATE", "This knowledge document was already imported", error);
        }
        throw new SqlTeacherException("KNOWLEDGE_DOCUMENT_IMPORT_FAILED", "Failed to import knowledge document", error);
    }
}

@Override
public List<KnowledgeDocument> listDocuments() {
    String sql = """
        select id, title, source_name, chunk_count, imported_at
        from knowledge_documents
        order by imported_at desc, title
        """;
    try (Connection connection = connectionFactory.open("app");
         PreparedStatement statement = connection.prepareStatement(sql);
         ResultSet rows = statement.executeQuery()) {
        List<KnowledgeDocument> documents = new ArrayList<>();
        while (rows.next()) {
            documents.add(new KnowledgeDocument(
                rows.getString("id"), rows.getString("title"), rows.getString("source_name"),
                rows.getInt("chunk_count"), Instant.parse(rows.getString("imported_at"))
            ));
        }
        return List.copyOf(documents);
    } catch (SQLException error) {
        throw new SqlTeacherException("KNOWLEDGE_DOCUMENT_LIST_FAILED", "Failed to list knowledge documents", error);
    }
}

@Override
public void deleteDocument(String documentId) {
    String id = requireText(documentId, "documentId");
    try (Connection connection = connectionFactory.open("app")) {
        connection.setAutoCommit(false);
        try (PreparedStatement deleteChunks = connection.prepareStatement(
            "delete from knowledge_chunks where document_id = ?"
```

```
        ); PreparedStatement deleteDocument = connection.prepareStatement(
            "delete from knowledge_documents where id = ?"
        )) {
            deleteChunks.setString(1, id);
            deleteChunks.executeUpdate();
            deleteDocument.setString(1, id);
            if (deleteDocument.executeUpdate() != 1) {
                throw new SqlTeacherException("KNOWLEDGE_DOCUMENT_NOT_FOUND", "Knowledge document not found");
            }
            connection.commit();
        } catch (SQLException | RuntimeException error) {
            connection.rollback();
            throw error;
        }
    } catch (SQLException error) {
        throw new SqlTeacherException("KNOWLEDGE_DOCUMENT_DELETE_FAILED", "Failed to delete knowledge document", error);
    }
}

@Override
public List<KnowledgeSearchResult> search(String requestedQuery, int limit) {
    String query = requireText(requestedQuery, "query");
    if (query.length() > 200) {
        throw new IllegalArgumentException("Knowledge search query must not exceed 200 characters");
    }
    if (limit < 1 || limit > 50) {
        throw new IllegalArgumentException("Knowledge search limit must be between 1 and 50");
    }
    String ftsQuery = toFtsQuery(query);
    String sql = """
        select d.id, d.title, d.source_name, c.chunk_index,
            snippet(knowledge_chunks_fts, 0, '【', '】', '…', 24) as matched_snippet,
            bm25(knowledge_chunks_fts) as score
        from knowledge_chunks_fts
        join knowledge_chunks c on c.rowid = knowledge_chunks_fts.rowid
        join knowledge_documents d on d.id = c.document_id
        where knowledge_chunks_fts match ?
        order by score, d.title, c.chunk_index
        limit ?
        """;
    try (Connection connection = connectionFactory.open("app");
         PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setString(1, ftsQuery);
        statement.setInt(2, limit);
        try (ResultSet rows = statement.executeQuery()) {
            List<KnowledgeSearchResult> results = new ArrayList<>();
            while (rows.next()) {
                results.add(new KnowledgeSearchResult(
                    rows.getString("id"), rows.getString("title"), rows.getString("source_name"),
                    rows.getInt("chunk_index"), rows.getString("matched_snippet"),
```

```
                    Math.max(0, -rows.getDouble("score"))
                ));
            }
            List<KnowledgeSearchResult> snapshot = List.copyOf(results);
            eventService.recordKnowledgeSearch(query.length(), snapshot.size());
            return snapshot;
        }
    } catch (SQLException error) {
        throw new SqlTeacherException("KNOWLEDGE_SEARCH_FAILED", "Failed to search local knowledge", error);
    }
}

static List<String> chunk(String content) {
    String normalized = content.replace("\r\n", "\n").replace('\r', '\n').trim();
    List<String> chunks = new ArrayList<>();
    StringBuilder current = new StringBuilder();
    for (String paragraph : normalized.split("\\n\\s*\\n")) {
        String clean = paragraph.replaceAll("[\\t ]+", " ").replaceAll("\\n+", "\n").trim();
        if (clean.isBlank()) {
            continue;
        }
        for (int offset = 0; offset < clean.length(); offset += MAX_CHUNK_CHARACTERS) {
            String part = clean.substring(offset, Math.min(clean.length(), offset + MAX_CHUNK_CHARACTERS));
            if (current.length() > 0 && current.length() + 2 + part.length() > MAX_CHUNK_CHARACTERS) {
                chunks.add(current.toString());
                current.setLength(0);
            }
            if (current.length() > 0) {
                current.append("\n\n");
            }
            current.append(part);
        }
    }
    if (current.length() > 0) {
        chunks.add(current.toString());
    }
    if (chunks.isEmpty()) {
        throw new IllegalArgumentException("Knowledge document has no searchable content");
    }
    return List.copyOf(chunks);
}

private static Path validatePath(Path requestedPath) {
    if (requestedPath == null) {
        throw new IllegalArgumentException("path must not be null");
    }
    Path path = requestedPath.toAbsolutePath().normalize();
    if (!Files.isRegularFile(path)) {
        throw new IllegalArgumentException("Knowledge document must be a regular file");
    }
```

```
    String name = path.getFileName().toString();
    int dot = name.lastIndexOf('.');
    String extension = dot < 0 ? "" : name.substring(dot + 1).toLowerCase(Locale.ROOT);
    if (!ALLOWED_EXTENSIONS.contains(extension)) {
        throw new IllegalArgumentException("Only UTF-8 .txt, .md, and .markdown documents are supported");
    }
    return path;
}

private static void insertDocument(
    Connection connection,
    String id,
    String title,
    String sourceName,
    String hash,
    int chunkCount,
    Instant importedAt
) throws SQLException {
    try (PreparedStatement statement = connection.prepareStatement("""
        insert into knowledge_documents(id, title, source_name, content_hash, chunk_count, imported_at)
        values (?, ?, ?, ?, ?, ?)
        """)) {
        statement.setString(1, id);
        statement.setString(2, title);
        statement.setString(3, sourceName);
        statement.setString(4, hash);
        statement.setInt(5, chunkCount);
        statement.setString(6, importedAt.toString());
        statement.executeUpdate();
    }
}

private static void insertChunks(Connection connection, String documentId, List<String> chunks) throws SQLException {
    try (PreparedStatement statement = connection.prepareStatement("""
        insert into knowledge_chunks(id, document_id, chunk_index, content) values (?, ?, ?, ?)
        """)) {
        for (int index = 0; index < chunks.size(); index++) {
            statement.setString(1, UUID.randomUUID().toString());
            statement.setString(2, documentId);
            statement.setInt(3, index);
            statement.setString(4, chunks.get(index));
            statement.addBatch();
        }
        statement.executeBatch();
    }
}

private static String title(Path path, String content) {
    for (String line : content.lines().limit(20).toList()) {
        String trimmed = line.trim();
```

```
        if (trimmed.startsWith("#")) {
            String heading = trimmed.replaceFirst("^#+\\s*", "").trim();
            if (!heading.isBlank()) {
                return truncate(heading, 160);
            }
        }
    }
    String fileName = path.getFileName().toString();
    int dot = fileName.lastIndexOf('.');
    return truncate(dot > 0 ? fileName.substring(0, dot) : fileName, 160);
}

private static String toFtsQuery(String query) {
    String[] tokens = query.trim().split("\\s+");
    List<String> phrases = new ArrayList<>();
    for (String token : tokens) {
        String safe = token.replace("\"", "\"\"").trim();
        if (!safe.isBlank()) {
            phrases.add("\"" + safe + "\"");
        }
    }
    if (phrases.isEmpty()) {
        throw new IllegalArgumentException("query must contain searchable text");
    }
    return String.join(" AND ", phrases);
}

private static String sha256(byte[] content) {
    try {
        return HexFormat.of().formatHex(MessageDigest.getInstance("SHA-256").digest(content));
    } catch (NoSuchAlgorithmException impossible) {
        throw new IllegalStateException("SHA-256 is unavailable", impossible);
    }
}

private static String decodeUtf8(byte[] content) {
    try {
        return StandardCharsets.UTF_8.newDecoder()
            .onMalformedInput(CodingErrorAction.REPORT)
            .onUnmappableCharacter(CodingErrorAction.REPORT)
            .decode(ByteBuffer.wrap(content))
            .toString();
    } catch (CharacterCodingException error) {
        throw new IllegalArgumentException("Knowledge document must use valid UTF-8", error);
    }
}

private static String truncate(String value, int maxLength) {
    return value.length() <= maxLength ? value : value.substring(0, maxLength);
}
```

```
    private static String requireText(String value, String name) {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException(name + " must not be blank");
        }
        return value.trim();
    }
}
// FILE: src/main/java/com/sqlteacher/application/event/DefaultLearningEventService.java
package com.sqlteacher.application.event;

import com.sqlteacher.application.risk.SqlRiskLevel;

import java.time.Clock;
import java.time.Duration;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Objects;

public final class DefaultLearningEventService implements LearningEventService {
    private final LearningEventRecorder recorder;
    private final Clock clock;
    private final LearningEventOwnerProvider ownerProvider;

    public DefaultLearningEventService(LearningEventRecorder recorder) {
        this(recorder, Clock.systemUTC(), () -> LearningEventOwnerProvider.GUEST_OWNER);
    }

    public DefaultLearningEventService(
        LearningEventRecorder recorder,
        LearningEventOwnerProvider ownerProvider
    ) {
        this(recorder, Clock.systemUTC(), ownerProvider);
    }

    DefaultLearningEventService(LearningEventRecorder recorder, Clock clock) {
        this(recorder, clock, () -> LearningEventOwnerProvider.GUEST_OWNER);
    }

    DefaultLearningEventService(
        LearningEventRecorder recorder,
        Clock clock,
        LearningEventOwnerProvider ownerProvider
    ) {
        this.recorder = Objects.requireNonNull(recorder, "recorder must not be null");
        this.clock = Objects.requireNonNull(clock, "clock must not be null");
        this.ownerProvider = Objects.requireNonNull(ownerProvider, "ownerProvider must not be null");
    }

    @Override
```

```
public void recordSqlExecution(
    String connectionId,
    boolean successful,
    String statementType,
    Duration duration,
    int resultCount,
    String errorCode
) {
    validateConnectionId(connectionId);
    validateText(statementType, "statementType");
    Objects.requireNonNull(duration, "duration must not be null");
    if (duration.isNegative()) {
        throw new IllegalArgumentException("duration must not be negative");
    }
    if (resultCount < 0) {
        throw new IllegalArgumentException("resultCount must not be negative");
    }

    Map<String, String> attributes = new LinkedHashMap<>();
    attributes.put("statementType", statementType);
    attributes.put("durationMs", Long.toString(duration.toMillis()));
    attributes.put("resultCount", Integer.toString(resultCount));
    putIfPresent(attributes, "errorCode", errorCode);
    record(LearningEventType.SQL_EXECUTION, connectionId, successful, attributes);
}

@Override
public void recordSqlRiskBlocked(
    String connectionId,
    String statementType,
    SqlRiskLevel riskLevel,
    boolean multiStatement
) {
    validateConnectionId(connectionId);
    validateText(statementType, "statementType");
    Objects.requireNonNull(riskLevel, "riskLevel must not be null");

    record(
        LearningEventType.SQL_RISK_BLOCKED,
        connectionId,
        false,
        Map.of(
            "statementType", statementType,
            "riskLevel", riskLevel.name(),
            "multiStatement", Boolean.toString(multiStatement)
        )
    );
}

@Override
```

```
public void recordAiGeneration(
    String connectionId,
    boolean successful,
    String model,
    String promptVersion,
    String errorCode
) {
    validateConnectionId(connectionId);
    validateText(model, "model");
    validateText(promptVersion, "promptVersion");

    Map<String, String> attributes = new LinkedHashMap<>();
    attributes.put("model", model);
    attributes.put("promptVersion", promptVersion);
    putIfPresent(attributes, "errorCode", errorCode);
    record(
        successful ? LearningEventType.AI_SQL_GENERATED : LearningEventType.AI_GENERATION_FAILED,
        connectionId,
        successful,
        attributes
    );
}

@Override
public void recordExerciseAttempt(
    String exerciseId,
    String status,
    boolean successful,
    Duration duration,
    String errorCode
) {
    validateText(exerciseId, "exerciseId");
    validateText(status, "status");
    Objects.requireNonNull(duration, "duration must not be null");
    if (duration.isNegative()) {
        throw new IllegalArgumentException("duration must not be negative");
    }
    Map<String, String> attributes = new LinkedHashMap<>();
    attributes.put("exerciseId", exerciseId);
    attributes.put("status", status);
    attributes.put("durationMs", Long.toString(duration.toMillis()));
    putIfPresent(attributes, "errorCode", errorCode);
    LearningEventType type = switch (status) {
        case "PASSED" -> LearningEventType.EXERCISE_PASSED;
        case "FAILED" -> LearningEventType.EXERCISE_FAILED;
        default -> LearningEventType.EXERCISE_ATTEMPT;
    };
    record(type, "exercise", successful, attributes);
}
```

```
@Override
public void recordExerciseHint(String exerciseId, int hintLevel) {
    validateText(exerciseId, "exerciseId");
    if (hintLevel < 1) {
        throw new IllegalArgumentException("hintLevel must be positive");
    }
    record(
        LearningEventType.EXERCISE_HINT_USED,
        "exercise",
        true,
        Map.of("exerciseId", exerciseId, "hintLevel", Integer.toString(hintLevel))
    );
}

@Override
public void recordKnowledgeSearch(int queryLength, int resultCount) {
    if (queryLength < 1 || resultCount < 0) {
        throw new IllegalArgumentException("knowledge search metrics are invalid");
    }
    record(
        LearningEventType.KNOWLEDGE_SEARCHED,
        "knowledge",
        true,
        Map.of("queryLength", Integer.toString(queryLength), "resultCount", Integer.toString(resultCount))
    );
}

private void record(
    LearningEventType type,
    String connectionId,
    boolean successful,
    Map<String, String> attributes
) {
    Map<String, String> ownedAttributes = new LinkedHashMap<>(attributes);
    ownedAttributes.put(LearningEventOwnerProvider.OWNER_ATTRIBUTE, ownerProvider.currentOwnerId());
    recorder.record(new LearningEvent(type, clock.instant(), connectionId, successful, ownedAttributes));
}

private static void validateConnectionId(String connectionId) {
    validateText(connectionId, "connectionId");
}

private static void validateText(String value, String name) {
    if (value == null || value.isBlank()) {
        throw new IllegalArgumentException(name + " must not be blank");
    }
}

private static void putIfPresent(Map<String, String> attributes, String name, String value) {
    if (value != null && !value.isBlank()) {
```

```
            attributes.put(name, value);
        }
    }
}
// FILE: src/main/java/com/sqlteacher/infrastructure/database/JdbcLearningAnalyticsService.java
package com.sqlteacher.infrastructure.database;

import com.sqlteacher.application.analytics.AnalyticsCsvExport;
import com.sqlteacher.application.analytics.AnalyticsFilter;
import com.sqlteacher.application.analytics.AnalyticsOverview;
import com.sqlteacher.application.analytics.ErrorAnalytics;
import com.sqlteacher.application.analytics.ExerciseAnalyticsRow;
import com.sqlteacher.application.analytics.KnowledgePointAnalytics;
import com.sqlteacher.application.analytics.LearningAnalyticsReport;
import com.sqlteacher.application.analytics.LearningAnalyticsService;
import com.sqlteacher.domain.SqlTeacherException;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

public final class JdbcLearningAnalyticsService implements LearningAnalyticsService {
    private static final DateTimeFormatter FILE_TIME = DateTimeFormatter.ofPattern("yyyyMMdd-HHmmss")
        .withZone(ZoneOffset.UTC);
    private final JdbcConnectionFactory connectionFactory;
    private final Clock clock;

    public JdbcLearningAnalyticsService(JdbcConnectionFactory connectionFactory) {
        this(connectionFactory, Clock.systemUTC());
    }

    JdbcLearningAnalyticsService(JdbcConnectionFactory connectionFactory, Clock clock) {
        this.connectionFactory = connectionFactory;
        this.clock = clock;
    }

    @Override
    public LearningAnalyticsReport analyze(AnalyticsFilter requestedFilter) {
```

```
    AnalyticsFilter filter = requestedFilter == null ? AnalyticsFilter.all() : requestedFilter;
    try (Connection connection = connectionFactory.open("app")) {
        List<ExerciseRecord> exercises = loadExercises(connection, filter);
        List<AttemptRecord> attempts = loadAttempts(connection, filter);
        int sessions = countSessions(connection, filter);
        return buildReport(filter, exercises, attempts, sessions, clock.instant());
    } catch (SQLException | IllegalArgumentException error) {
        throw new SqlTeacherException("LEARNING_ANALYTICS_READ_FAILED", "Failed to calculate learning analytics", error);
    }
}

@Override
public AnalyticsCsvExport exportCsv(AnalyticsFilter filter) {
    LearningAnalyticsReport report = analyze(filter);
    StringBuilder csv = new StringBuilder("\ufeff");
    csv.append("SQLTeacher 学情导出\r\n");
    row(csv, "生成时间(UTC)", report.generatedAt().toString());
    row(csv, "开始时间(含)", value(report.filter().startInclusive()));
    row(csv, "结束时间(不含)", value(report.filter().endExclusive()));
    row(csv, "题目筛选", value(report.filter().exerciseId()));
    row(csv, "知识点筛选", value(report.filter().knowledgePoint()));
    row(csv, "错误类型筛选", value(report.filter().errorCode()));
    csv.append("统计口径,通过率=通过提交/全部提交;完成率=筛选范围内至少通过一次的题目/启用题目;平均尝试=尝试数/已完成题目\r\n\r\n");
    csv.append("会话,尝试,提交,通过提交,通过率,平均尝试,平均提交耗时(ms),完成题目,题目总数,完成率\r\n");
    AnalyticsOverview overview = report.overview();
    row(csv, overview.sessions(), overview.attempts(), overview.submissions(), overview.passedSubmissions(),
        decimal(overview.passRate()), decimal(overview.averageAttemptsPerCompletedExercise()),
        overview.averageSubmissionDuration().toMillis(), overview.completedExercises(), overview.totalExercises(),
        decimal(overview.completionRate()));
    csv.append("\r\n题目 ID,题目,知识点,尝试,提交,通过提交,失败提交,通过率,是否完成,最后尝试时间(UTC)\r\n");
    for (ExerciseAnalyticsRow item : report.exercises()) {
        row(csv, item.exerciseId(), item.title(), item.knowledgePoint(), item.attempts(), item.submissions(),
            item.passedSubmissions(), item.failedSubmissions(), decimal(item.passRate()),
            item.completed() ? "是" : "否", item.lastAttempt().map(Instant::toString).orElse(""));
    }
    csv.append("\r\n错误类型,次数\r\n");
    for (ErrorAnalytics error : report.commonErrors()) {
        row(csv, error.errorCode(), error.count());
    }
    csv.append("\r\n知识点,尝试,失败提交,完成题目,题目总数,薄弱度\r\n");
    for (KnowledgePointAnalytics point : report.knowledgePoints()) {
        row(csv, point.knowledgePoint(), point.attempts(), point.failedSubmissions(),
            point.completedExercises(), point.totalExercises(), decimal(point.weaknessRate()));
    }
    return new AnalyticsCsvExport(
        "sqlteacher-analytics-" + FILE_TIME.format(report.generatedAt()) + ".csv",
        csv.toString(),
        report.generatedAt()
    );
}
```

```
private static List<ExerciseRecord> loadExercises(Connection connection, AnalyticsFilter filter) throws SQLException {
    StringBuilder sql = new StringBuilder("select id, title, knowledge_point from exercises where enabled = 1");
    List<String> parameters = new ArrayList<>();
    if (filter.exerciseId() != null) {
        sql.append(" and id = ?");
        parameters.add(filter.exerciseId());
    }
    if (filter.knowledgePoint() != null) {
        sql.append(" and knowledge_point = ?");
        parameters.add(filter.knowledgePoint());
    }
    sql.append(" order by knowledge_point, title");
    try (PreparedStatement statement = connection.prepareStatement(sql.toString())) {
        bind(statement, parameters);
        try (ResultSet rows = statement.executeQuery()) {
            List<ExerciseRecord> result = new ArrayList<>();
            while (rows.next()) {
                result.add(new ExerciseRecord(
                    rows.getString("id"), rows.getString("title"), rows.getString("knowledge_point")
                ));
            }
            return List.copyOf(result);
        }
    }
}

private static List<AttemptRecord> loadAttempts(Connection connection, AnalyticsFilter filter) throws SQLException {
    StringBuilder sql = new StringBuilder("""
        select a.id, a.session_id, s.exercise_id, a.status, a.duration_ms, a.error_code, a.created_at
        from exercise_attempts a
        join exercise_sessions s on s.id = a.session_id
        join exercises e on e.id = s.exercise_id
        where e.enabled = 1
        """);
    List<String> parameters = new ArrayList<>();
    appendFilters(sql, parameters, filter, "a.created_at", true);
    sql.append(" order by a.created_at, a.id");
    try (PreparedStatement statement = connection.prepareStatement(sql.toString())) {
        bind(statement, parameters);
        try (ResultSet rows = statement.executeQuery()) {
            List<AttemptRecord> result = new ArrayList<>();
            while (rows.next()) {
                result.add(new AttemptRecord(
                    rows.getString("id"), rows.getString("session_id"), rows.getString("exercise_id"),
                    rows.getString("status"), rows.getLong("duration_ms"), rows.getString("error_code"),
                    Instant.parse(rows.getString("created_at"))
                ));
            }
            return List.copyOf(result);
```

```
        }
    }
}

private static int countSessions(Connection connection, AnalyticsFilter filter) throws SQLException {
    StringBuilder sql = new StringBuilder("""
        select count(distinct s.id)
        from exercise_sessions s
        join exercises e on e.id = s.exercise_id
        where e.enabled = 1
        """);
    List<String> parameters = new ArrayList<>();
    appendFilters(sql, parameters, filter, "s.started_at", false);
    if (filter.errorCode() != null) {
        sql.append(" and exists (select 1 from exercise_attempts ae where ae.session_id = s.id and ae.error_code = ?)");
        parameters.add(filter.errorCode());
    }
    try (PreparedStatement statement = connection.prepareStatement(sql.toString())) {
        bind(statement, parameters);
        try (ResultSet row = statement.executeQuery()) {
            row.next();
            return row.getInt(1);
        }
    }
}

private static void appendFilters(
    StringBuilder sql,
    List<String> parameters,
    AnalyticsFilter filter,
    String timeColumn,
    boolean includeError
) {
    if (filter.startInclusive() != null) {
        sql.append(" and ").append(timeColumn).append(" >= ?");
        parameters.add(filter.startInclusive().toString());
    }
    if (filter.endExclusive() != null) {
        sql.append(" and ").append(timeColumn).append(" < ?");
        parameters.add(filter.endExclusive().toString());
    }
    if (filter.exerciseId() != null) {
        sql.append(" and e.id = ?");
        parameters.add(filter.exerciseId());
    }
    if (filter.knowledgePoint() != null) {
        sql.append(" and e.knowledge_point = ?");
        parameters.add(filter.knowledgePoint());
    }
    if (includeError && filter.errorCode() != null) {
```

```
        sql.append(" and a.error_code = ?");
        parameters.add(filter.errorCode());
    }
}

private static LearningAnalyticsReport buildReport(
    AnalyticsFilter filter,
    List<ExerciseRecord> exercises,
    List<AttemptRecord> attempts,
    int sessions,
    Instant generatedAt
) {
    Map<String, ExerciseAccumulator> byExercise = new LinkedHashMap<>();
    for (ExerciseRecord exercise : exercises) {
        byExercise.put(exercise.id(), new ExerciseAccumulator(exercise));
    }
    Map<String, Integer> errors = new LinkedHashMap<>();
    long submissionDuration = 0;
    int submissions = 0;
    int passedSubmissions = 0;
    for (AttemptRecord attempt : attempts) {
        ExerciseAccumulator accumulator = byExercise.get(attempt.exerciseId());
        if (accumulator == null) {
            continue;
        }
        accumulator.accept(attempt);
        if (attempt.submission()) {
            submissions++;
            submissionDuration += attempt.durationMs();
            if (attempt.passed()) {
                passedSubmissions++;
            }
        }
        if (attempt.errorCode() != null && !attempt.errorCode().isBlank()) {
            errors.merge(attempt.errorCode(), 1, Integer::sum);
        }
    }
    List<ExerciseAnalyticsRow> exerciseRows = byExercise.values().stream()
        .map(ExerciseAccumulator::toRow)
        .sorted(Comparator.comparing(ExerciseAnalyticsRow::completed)
            .thenComparing(ExerciseAnalyticsRow::failedSubmissions, Comparator.reverseOrder())
            .thenComparing(ExerciseAnalyticsRow::title))
        .toList();
    int completed = (int) exerciseRows.stream().filter(ExerciseAnalyticsRow::completed).count();
    AnalyticsOverview overview = new AnalyticsOverview(
        sessions,
        attempts.size(),
        submissions,
        passedSubmissions,
        ratio(passedSubmissions, submissions),
```

```
        completed == 0 ? 0 : (double) attempts.size() / completed,
        Duration.ofMillis(submissions == 0 ? 0 : Math.round((double) submissionDuration / submissions)),
        completed,
        exercises.size(),
        ratio(completed, exercises.size())
    );
    List<ErrorAnalytics> errorRows = errors.entrySet().stream()
        .map(entry -> new ErrorAnalytics(entry.getKey(), entry.getValue()))
        .sorted(Comparator.comparingInt(ErrorAnalytics::count).reversed().thenComparing(ErrorAnalytics::errorCode))
        .toList();
    List<KnowledgePointAnalytics> knowledgeRows = buildKnowledgeRows(exerciseRows);
    return new LearningAnalyticsReport(filter, generatedAt, overview, exerciseRows, errorRows, knowledgeRows);
}

private static List<KnowledgePointAnalytics> buildKnowledgeRows(List<ExerciseAnalyticsRow> exercises) {
    Map<String, List<ExerciseAnalyticsRow>> grouped = new LinkedHashMap<>();
    for (ExerciseAnalyticsRow exercise : exercises) {
        grouped.computeIfAbsent(exercise.knowledgePoint(), ignored -> new ArrayList<>()).add(exercise);
    }
    return grouped.entrySet().stream().map(entry -> {
        List<ExerciseAnalyticsRow> rows = entry.getValue();
        int attempts = rows.stream().mapToInt(ExerciseAnalyticsRow::attempts).sum();
        int failures = rows.stream().mapToInt(ExerciseAnalyticsRow::failedSubmissions).sum();
        int submissions = rows.stream().mapToInt(ExerciseAnalyticsRow::submissions).sum();
        int completed = (int) rows.stream().filter(ExerciseAnalyticsRow::completed).count();
        double failureRate = ratio(failures, submissions);
        double incompleteRate = 1 - ratio(completed, rows.size());
        double weakness = Math.min(1, failureRate * 0.6 + incompleteRate * 0.4);
        return new KnowledgePointAnalytics(entry.getKey(), attempts, failures, completed, rows.size(), weakness);
    }).sorted(Comparator.comparingDouble(KnowledgePointAnalytics::weaknessRate).reversed()
        .thenComparing(KnowledgePointAnalytics::knowledgePoint)).toList();
}

private static void bind(PreparedStatement statement, List<String> parameters) throws SQLException {
    for (int index = 0; index < parameters.size(); index++) {
        statement.setString(index + 1, parameters.get(index));
    }
}

private static double ratio(int numerator, int denominator) {
    return denominator == 0 ? 0 : (double) numerator / denominator;
}

private static void row(StringBuilder csv, Object... values) {
    for (int index = 0; index < values.length; index++) {
        if (index > 0) {
            csv.append(',');
        }
        csv.append(escape(String.valueOf(values[index])));
    }
```

```
    csv.append("\r\n");
}

private static String escape(String value) {
    String normalized = value.replace("\r", " ").replace("\n", " ");
    return '"' + normalized.replace("\"", "\"\"") + '"';
}

private static String value(Object value) {
    return value == null ? "全部" : value.toString();
}

private static String decimal(double value) {
    return String.format(Locale.ROOT, "%.4f", value);
}

private record ExerciseRecord(String id, String title, String knowledgePoint) {
}

private record AttemptRecord(
    String id,
    String sessionId,
    String exerciseId,
    String status,
    long durationMs,
    String errorCode,
    Instant createdAt
) {
    boolean submission() {
        return "PASSED".equals(status) || "FAILED".equals(status);
    }

    boolean passed() {
        return "PASSED".equals(status);
    }
}

private static final class ExerciseAccumulator {
    private final ExerciseRecord exercise;
    private int attempts;
    private int submissions;
    private int passed;
    private int failed;
    private Instant lastAttempt;

    private ExerciseAccumulator(ExerciseRecord exercise) {
        this.exercise = exercise;
    }

    private void accept(AttemptRecord attempt) {
```

```
            attempts++;
            if (attempt.submission()) {
                submissions++;
                if (attempt.passed()) {
                    passed++;
                } else {
                    failed++;
                }
            }
            if (lastAttempt == null || attempt.createdAt().isAfter(lastAttempt)) {
                lastAttempt = attempt.createdAt();
            }
        }

        private ExerciseAnalyticsRow toRow() {
            return new ExerciseAnalyticsRow(
                exercise.id(), exercise.title(), exercise.knowledgePoint(), attempts, submissions, passed, failed,
                ratio(passed, submissions), passed > 0, lastAttempt
            );
        }
    }
}
// FILE: src/main/java/com/sqlteacher/infrastructure/database/SqliteApplicationBackupService.java
package com.sqlteacher.infrastructure.database;

import com.sqlteacher.application.config.SqlTeacherConfiguration;
import com.sqlteacher.application.maintenance.ApplicationBackupService;
import com.sqlteacher.application.maintenance.BackupSnapshot;
import com.sqlteacher.domain.SqlTeacherException;

import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.file.AtomicMoveNotSupportedException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.nio.file.attribute.FileTime;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.time.Instant;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.UUID;
```

```
import java.util.zip.ZipEntry;
import java.util.zip.ZipInputStream;
import java.util.zip.ZipOutputStream;

public final class SqliteApplicationBackupService implements ApplicationBackupService {
    private static final DateTimeFormatter ID_TIME =
        DateTimeFormatter.ofPattern("yyyyMMdd-HHmmss").withZone(ZoneOffset.UTC);
    private static final int MAX_BACKUPS = 20;

    private final SqlTeacherConfiguration configuration;
    private final Path backupDirectory;

    public SqliteApplicationBackupService(SqlTeacherConfiguration configuration) {
        this.configuration = configuration;
        this.backupDirectory = configuration.dataDirectory().resolve("backups").toAbsolutePath().normalize();
    }

    @Override
    public BackupSnapshot createBackup() {
        return createBackup(false, "manual");
    }

    BackupSnapshot createAutomaticBackup(String reason) {
        String normalizedReason = reason == null ? "upgrade" : reason.replaceAll("[^a-zA-Z0-9-]", "-");
        return createBackup(true, "auto-" + normalizedReason);
    }

    @Override
    public List<BackupSnapshot> listBackups() {
        if (Files.notExists(backupDirectory)) {
            return List.of();
        }
        try (var paths = Files.list(backupDirectory)) {
            return paths
                .filter(path -> path.getFileName().toString().endsWith(".zip"))
                .map(this::snapshot)
                .sorted(Comparator.comparing(BackupSnapshot::createdAt).reversed())
                .toList();
        } catch (IOException error) {
            throw failure("BACKUP_LIST_FAILED", "Failed to list application backups", error);
        }
    }

    @Override
    public void restoreBackup(String backupId) {
        Path archive = resolveBackup(backupId);
        if (Files.notExists(archive)) {
            throw new SqlTeacherException("BACKUP_NOT_FOUND", "The selected backup no longer exists");
        }
        Path restoreDirectory = configuration.dataDirectory().resolve(".restore-" + UUID.randomUUID()).normalize();
```

```
    String safetyBackupId = null;
    boolean replacementStarted = false;
    try {
        Files.createDirectories(restoreDirectory);
        extractDatabaseFiles(archive, restoreDirectory);
        Path restoredApp = restoreDirectory.resolve("app.db");
        Path restoredDemo = restoreDirectory.resolve("demo.db");
        if (Files.notExists(restoredApp)) {
            throw new IOException("Backup does not contain app.db");
        }
        verifyIntegrity(restoredApp);
        if (Files.exists(restoredDemo)) {
            verifyIntegrity(restoredDemo);
        }

        safetyBackupId = createAutomaticBackup("before-restore").id();
        replacementStarted = true;
        replaceDatabase(restoredApp, configuration.database().appDatabasePath());
        if (Files.exists(restoredDemo)) {
            replaceDatabase(restoredDemo, configuration.database().demoDatabasePath());
        }
    } catch (IOException | SQLException error) {
        if (replacementStarted && safetyBackupId != null) {
            try {
                Path rollbackDirectory = restoreDirectory.resolve("rollback");
                Files.createDirectories(rollbackDirectory);
                extractDatabaseFiles(resolveBackup(safetyBackupId), rollbackDirectory);
                replaceDatabase(rollbackDirectory.resolve("app.db"), configuration.database().appDatabasePath());
                Path rollbackDemo = rollbackDirectory.resolve("demo.db");
                if (Files.exists(rollbackDemo)) {
                    replaceDatabase(rollbackDemo, configuration.database().demoDatabasePath());
                }
            } catch (IOException rollbackError) {
                error.addSuppressed(rollbackError);
            }
        }
        throw failure("BACKUP_RESTORE_FAILED", "Failed to restore the selected backup", error);
    } finally {
        deleteTreeQuietly(restoreDirectory);
    }
}

@Override
public void restoreDemoDatabase() {
    Path staged = configuration.dataDirectory().resolve(".demo-restore-" + UUID.randomUUID() + ".db");
    try {
        SqliteAppDatabaseInitializer.createDemoDatabase(staged, true);
        verifyIntegrity(staged);
        replaceDatabase(staged, configuration.database().demoDatabasePath());
    } catch (IOException | SQLException error) {
```

```
        throw failure("DEMO_RESTORE_FAILED", "Failed to restore the demonstration database", error);
    } finally {
        try {
            Files.deleteIfExists(staged);
        } catch (IOException ignored) {
            // The staged file is harmless and will be replaced on the next restore attempt.
        }
    }
}

private BackupSnapshot createBackup(boolean automatic, String prefix) {
    Instant now = Instant.now();
    String id = prefix + "-" + ID_TIME.format(now) + "-" + UUID.randomUUID().toString().substring(0, 8);
    Path archive = resolveBackup(id);
    Path staging = configuration.dataDirectory().resolve(".backup-" + UUID.randomUUID()).normalize();
    try {
        Files.createDirectories(backupDirectory);
        Files.createDirectories(staging);
        Path appCopy = staging.resolve("app.db");
        Path demoCopy = staging.resolve("demo.db");
        createConsistentCopy(configuration.database().appDatabasePath(), appCopy);
        createConsistentCopy(configuration.database().demoDatabasePath(), demoCopy);
        if (Files.notExists(appCopy)) {
            throw new IOException("Application database does not exist");
        }
        try (OutputStream output = Files.newOutputStream(archive);
             ZipOutputStream zip = new ZipOutputStream(output)) {
            addToArchive(zip, appCopy, "app.db");
            if (Files.exists(demoCopy)) {
                addToArchive(zip, demoCopy, "demo.db");
            }
        }
        pruneOldBackups();
        return new BackupSnapshot(id, now, Files.size(archive), automatic);
    } catch (IOException | SQLException error) {
        try {
            Files.deleteIfExists(archive);
        } catch (IOException ignored) {
            // Preserve the original failure.
        }
        throw failure("BACKUP_CREATE_FAILED", "Failed to create an application backup", error);
    } finally {
        deleteTreeQuietly(staging);
    }
}

private static void createConsistentCopy(Path source, Path target) throws SQLException, IOException {
    if (Files.notExists(source)) {
        return;
    }
```

```
    Files.createDirectories(target.toAbsolutePath().getParent());
    SqliteDriver.ensureLoaded();
    try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + source.toAbsolutePath());
         Statement statement = connection.createStatement()) {
        String escaped = target.toAbsolutePath().toString().replace("'", "''");
        statement.execute("vacuum into '" + escaped + "'");
    }
}

private static void verifyIntegrity(Path database) throws SQLException {
    SqliteDriver.ensureLoaded();
    try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + database.toAbsolutePath());
         Statement statement = connection.createStatement();
         ResultSet result = statement.executeQuery("pragma integrity_check")) {
        if (!result.next() || !"ok".equalsIgnoreCase(result.getString(1))) {
            throw new SQLException("SQLite integrity check failed for restored database");
        }
    }
}

private static void addToArchive(ZipOutputStream zip, Path source, String name) throws IOException {
    zip.putNextEntry(new ZipEntry(name));
    Files.copy(source, zip);
    zip.closeEntry();
}

private static void extractDatabaseFiles(Path archive, Path destination) throws IOException {
    try (InputStream input = Files.newInputStream(archive);
         ZipInputStream zip = new ZipInputStream(input)) {
        ZipEntry entry;
        while ((entry = zip.getNextEntry()) != null) {
            if (!entry.isDirectory() && ("app.db".equals(entry.getName()) || "demo.db".equals(entry.getName()))) {
                Files.copy(zip, destination.resolve(entry.getName()), StandardCopyOption.REPLACE_EXISTING);
            }
            zip.closeEntry();
        }
    }
}

private static void replaceDatabase(Path source, Path destination) throws IOException {
    Files.createDirectories(destination.toAbsolutePath().getParent());
    Path staged = destination.resolveSibling(destination.getFileName() + ".restore-staged");
    Files.copy(source, staged, StandardCopyOption.REPLACE_EXISTING);
    try {
        Files.move(staged, destination, StandardCopyOption.ATOMIC_MOVE, StandardCopyOption.REPLACE_EXISTING);
    } catch (AtomicMoveNotSupportedException ignored) {
        Files.move(staged, destination, StandardCopyOption.REPLACE_EXISTING);
    }
}
```

```
    private Path resolveBackup(String backupId) {
        if (backupId == null || !backupId.matches("[a-zA-Z0-9._-]+")) {
            throw new SqlTeacherException("BACKUP_ID_INVALID", "Invalid backup identifier");
        }
        Path resolved = backupDirectory.resolve(backupId + ".zip").normalize();
        if (!resolved.getParent().equals(backupDirectory)) {
            throw new SqlTeacherException("BACKUP_ID_INVALID", "Invalid backup identifier");
        }
        return resolved;
    }

    private BackupSnapshot snapshot(Path archive) {
        try {
            String fileName = archive.getFileName().toString();
            String id = fileName.substring(0, fileName.length() - 4);
            FileTime modified = Files.getLastModifiedTime(archive);
            return new BackupSnapshot(id, modified.toInstant(), Files.size(archive), id.startsWith("auto-"));
        } catch (IOException error) {
            throw failure("BACKUP_READ_FAILED", "Failed to read application backup metadata", error);
        }
    }

    private void pruneOldBackups() throws IOException {
        List<BackupSnapshot> snapshots = new ArrayList<>(listBackups());
        for (int index = MAX_BACKUPS; index < snapshots.size(); index++) {
            Files.deleteIfExists(resolveBackup(snapshots.get(index).id()));
        }
    }

    private static void deleteTreeQuietly(Path root) {
        if (root == null || Files.notExists(root)) {
            return;
        }
        try (var paths = Files.walk(root)) {
            paths.sorted(Comparator.reverseOrder()).forEach(path -> {
                try {
                    Files.deleteIfExists(path);
                } catch (IOException ignored) {
                    // Best-effort cleanup only.
                }
            });
        } catch (IOException ignored) {
            // Best-effort cleanup only.
        }
    }

    private static SqlTeacherException failure(String code, String message, Exception cause) {
        return new SqlTeacherException(code, message, cause);
    }
}
```